

Robustness-Guaranteed Reinforcement Learning Under Uncertainties in Dynamics Modeling and State Estimates

Duofeng Pan , Yongquan Huang , Rong Chen , Wenjie Lu , *Member, IEEE*, and Manman Hu , *Member, IEEE*

Abstract—Robust reinforcement learning enhances system robustness against uncertainties through bilevel optimization. However, the controller learned by reinforcement learning struggles to offer a strict robustness guarantee, limiting its reliability in real-world applications. This article presents robustness-guaranteed reinforcement learning framework under bounded uncertainties in dynamics modeling and state estimates. By describing the uncertainties and system dynamics models via ReLU neural networks, the states that most violate the proposed robustness conditions can be exactly found, and their robustness is prioritized during training. The solvability of the reinforcement learning problem is particularly studied, yielding a necessary condition that captures the adversarial relationship between control saturation and uncertainties. Subsequently, the attractive region can be prevalidated and the maximum robustness of the controller can be achieved by amplifying the uncertainties to a derived maximum extent in training. The proposed method has been fully evaluated in simulation and its advantages have been proven, including 3-D quadrotor reference trajectory tracking tasks.

Impact Statement—This article proposes an innovative approach to strengthen the robustness of reinforcement learning (RL) algorithms in uncertain environments. Conventional RL methods are inherently limited by their sensitivity to uncertainties in system dynamics modeling and state estimation. To address these limitations, we introduce a novel methodology that integrates explicit uncertainty modeling with ReLU neural networks within a bilevel optimization framework. This approach guarantees robustness by ensuring that learned controllers perform well even with uncertainties. Unlike existing methods that yield

only approximate robustness guarantees, such as domain randomization, this approach provides strict guarantees. Extensive simulation results demonstrate its superiority in maintaining reliability under various uncertainty scenarios. Furthermore, its successful application in high-dimensional systems highlights the method's versatility and scalability. This research contributes theoretical advancements toward developing more reliable RL-based controllers for uncertain environments, with significant implications for autonomous systems and robotics applications.

Index Terms—Guaranteed robustness, piecewise linear, robust reinforcement learning, uncertain system.

NOMENCLATURE

$\bar{x}_t$	Estimation of the system state x_t .
$\bar{f}$	Approximation of the dynamics model f .
$\bar{u}_t$	Evaluation of the controller at $\bar{x}_t$.
$\bar{x}^*$	Equilibrium state.
x^c	Counter-example that violates the robust conditions.
σ	Uncertainty in dynamics modeling.
ϵ	Uncertainty in state estimates.
$\mathcal{D}$	Attractive region of the system.
$\mathcal{D}'$	Refined attractive region of the system.
$\Delta\sigma, \Delta\epsilon$	Bounds of the uncertainties σ, ϵ .
$\mathcal{G}_x V(x)$	Subgradient of Lyapunov function $V(x)$ to state variable x .
θ	Set of all trainable parameters of controller and Lyapunov networks.
θ	Half-angle of cone.
$\tilde{\mathcal{C}}$	Symmetric cones.
$\tilde{\eta}(x)$	A stack of the basis functions in the form of HL-PWL.
$\mathcal{P}(x)$	Upper bounds of $\mathcal{G}_x \pi(x)$.
$\mathcal{F}(x)$	Upper bounds of $\mathcal{G}_u f(x, u)$.
$\mathcal{V}$	The set of states at partition vertices.
$\tilde{\mathcal{U}}$	The admissible control set that satisfies the necessary condition.
$\mathcal{H}$	Boundary of $\tilde{\mathcal{U}}$.

I. INTRODUCTION

ROBOTIC systems are susceptible to stability degradation and catastrophic failures caused by various types of uncertainties in dynamics modeling and state estimates. Existing literature includes robust control [1], adaptive control [2],

Received 23 December 2024; revised 19 April 2025, 24 June 2025, 9 October 2025, and 9 February 2026; accepted 16 March 2026. Date of publication 20 March 2026; date of current version 31 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62003110; in part by Shenzhen Science and Technology Innovation Foundations under Grant JCYJ20210324132607018, Grant JSGG20210420091804012, and Grant GXWD20220811163649003; in part by the Science Center Program of the National Natural Science Foundation of China under Grant 62188101; and in part by the Open Research Fund of Guangdong Key Laboratory of Modern Control Technology. This article was recommended for publication by Associate Editor Peng Shi upon evaluation of the reviewers' comments. (*Corresponding author: Wenjie Lu.*)

Duofeng Pan, Yongquan Huang, Rong Chen, and Wenjie Lu are with Guangdong Provincial Key Laboratory of Intelligent Morphing Mechanisms and Adaptive Robotics, the School of Robotics and Advanced Manufacture, Harbin Institute of Technology (Shenzhen), Shenzhen 518055, China (e-mail: duofengpan@163.com; rongchen_2023@163.com; yq_huang2000@163.com; luwenjie@hit.edu.cn).

Manman Hu is with the Department of Civil Engineering, University of Hong Kong, Hong Kong 999077, China (e-mail: mmhu@hku.hk).

Digital Object Identifier 10.1109/TAI.2026.3675539

stochastic control [3], sliding mode control [4], and H-infinity [5]. For instance, a robust H-infinity Control method employs a feedback linearized controller to eliminate model nonlinearity, thereby effectively applying linear control methods [6]. Robust LQR has also made progress in capturing noise variability [7] and minimizing costs in worst-case scenarios [8]. Robust iLQR demonstrates effectiveness in solving many complex nonlinear system optimization control problems [9], [10]. However, these approaches may be limited by the canonical models used in describing the uncertainties and dynamics.

Reinforcement learning (RL) algorithms rely on neural networks and offer sufficient flexibility in modeling uncertainties and dynamics. Robust RL enhances controller robustness by solving min-max problems or robust Markov decision processes (MDPs). Min-max optimizations maximize the worst-case reward, including robust actor-critic algorithms [11], saddle-point gradient methods [12], and adversarial RL [13], [14], [15], [16]. Robust MDP maximizes the expected rewards over all possible realizations of Markovian processes [17], including multiobjective optimization [18], Wasserstein robust MDP [19], and conditional-value-at-risk optimizations [20]. For example, domain randomization RL varies the parameters over a set of distributions and obtains a controller robust to these parameter variations [21]. In addition, to address worst-case observational attacks, a novel defense-aware robust RL approach has been proposed [22]. This approach employs an adversarial attacker to simulate severe observation perturbations while a robust defender aids the RL agent in learning optimal policies within set boundaries. Furthermore, a hierarchical framework merging high-frequency DRL with low-frequency MPC was developed to enhance control efficiency in uncertain conditions [23]. When paired with nonlinear MPC as an approximator, DRL can yield optimal policies for real systems, including those with stochastic dynamics [24]. Notably, Cohen et al. [25] presented a framework enabling the use of simplified models in constructing safety-critical controllers, offering innovative ideas for ensuring the safety of complex full-order models.

Existing RL approaches often fail to provide theoretical guarantees of closed-loop stability for controlled systems operating under uncertainties. This lack of robustness undermines the reliability of the learned controller, thereby restricting its practical applicability in safety-critical domains. Furthermore, many RL frameworks presume the existence of a predefined region of attraction without formal verification, potentially rendering the RL problem infeasible due to inherent contradictions in the problem setup. Consequently, the loss function minimized during training may be dominated by unsolvable states, leading to misleading backpropagation signals. This article extends prior work of ensuring stability through learning auxiliary Lyapunov functions [3] and presents robustness-guaranteed RL (RGRL) under bounded uncertainties in dynamics modeling and state estimates in a unified manner. As illustrated in Fig. 1, the dynamics and uncertainties are approximated by ReLU neural networks and can further be reformulated in an equivalent high-level piecewise linear (HL-PWL) representation. The HL-PWL representation and the robustness conditions yield the necessary

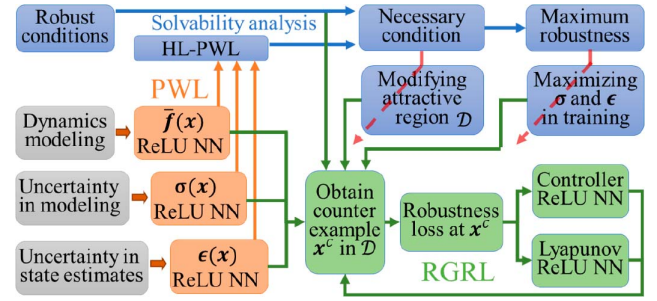


Fig. 1. Proposed robustness-guaranteed RL (RGRL) learns the robust controller and the Lyapunov function simultaneously. Based on the high-level piecewise linear (HL-PWL) representation of the dynamics model and uncertainties, through analyzing the solvability of the RL problem, the necessary conditions and maximum robustness are derived to improve the attractive region and the robustness capacity.

conditions for the existence of a robust controller. There are three main contributions.

1) A set of novel robustness conditions is designed to efficiently identify states that violate the conditions. These conditions are further enforced during the synthesized training of the Lyapunov network and the controller network.

2) This article proposes a necessary condition for the existence of both a robust controller and a Lyapunov function. This condition can be used to prevalidate or refine the attractive region before the RL learning process, ensuring the solvability of the RL problem.

3) This article introduces a geometric interpretation to demonstrate the existence of solutions to the robust RL problem, yielding the derivation of the maximum robustness under control saturations.

A number of numerical simulations on the inverted pendulum and quadrotors have demonstrated the robustness of the controllers obtained from the proposed RGRL, where multiplicative and additive uncertainties were considered. The comparisons against classical robustness methods have also shown the effectiveness of the learning-based method. The application of the controller in 3-D quadrotor systems has preliminarily demonstrated the effectiveness of the proposed method in high-dimensional systems.

In the following sections, related works are summarized in Section II. The control problem of dynamical systems under uncertainties is presented in Section III. Robustness conditions under various uncertainties are introduced in Section V, followed by the proposed RGRL in Section IV. The solvability analysis on the RL problem is given in Section VI. At last, the conclusion and future work are discussed in Section VIII.

II. RELATED WORK

Stability-guaranteed RL ensures the convergence of controllers by incorporating stability criteria into the RL framework, such as Lyapunov based methods [26], barrier functions [27], [28], control-theoretic methods [29], and safety exploration techniques [30]. A method based on Lyapunov theory employs neural networks as function approximators to address challenges in reinforcement learning. The controller action set

is constrained by Lyapunov stability theory, and the cost function serves as a Lyapunov candidate function to ensure system stability [31]. To further enhance the performance and theoretical guarantees of neural network controllers in robotic tasks, a neural network controller design method incorporating Lyapunov stability theory is proposed. Stability is verified through mixed-integer linear programming (MIP) and the attractive region of the closed-loop system is optimized [32]. Despite these advances, neural network methods still encounter challenges related to scalability and limited interpretability. A framework comprising a trainer for generating candidate Lyapunov functions and a validator for verifying these candidates has been developed. Its effectiveness on high-dimensional nonlinear dynamical systems has been specifically validated through RL training [33].

The solvability of RL problems is determined by control saturation, uncertainties and system dynamics in the attractive region. In [34], the uncertainty set is defined to be centering at a misspecified MDP. The unknown uncertainty set can be estimated from a single sampled trajectory [35]. It can be modeled as beliefs [36], [37]. Task-level components can be abstracted by bisimulation loss [38]. Confidence levels are used to penalize out-of-distribution samples [39]. Similar to H-infinity control, the small gain theorem is used to minimize the sensitivity of the stabilization against uncertainties [40]. However, strict robustness can not be guaranteed by these methods.

III. PROBLEM FORMULATION

Consider a discrete-time dynamical system and its dynamics model is given as

$$\mathbf{x}_{t+1} = \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t), \quad \mathbf{u}_{\min} \preceq \mathbf{u}_t \preceq \mathbf{u}_{\max} \quad (1)$$

where $\preceq$ denotes element-wise “less or equal”, $\mathbf{x}_t \in \mathbb{R}^{n_x}$ and $\mathbf{u}_t \in \mathbb{R}^{n_u}$ denote the states and control inputs, respectively. The control saturates at $\mathbf{u}_{\min}$ or $\mathbf{u}_{\max}$.

The dynamics model $\mathbf{f}(\mathbf{x}_t, \mathbf{u}_t)$ is hardly known exactly. Its approximation $\bar{\mathbf{f}}$ is assumed available through system identification and is given as

$$\begin{aligned} \mathbf{x}_{t+1} &= \bar{\mathbf{f}}(\mathbf{x}_t, \mathbf{u}_t) (\mathbf{1} + \boldsymbol{\sigma}_m) + \boldsymbol{\sigma}_a \\ &= \bar{\mathbf{f}}(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\sigma}(\mathbf{x}_t) \\ \mathbf{u}_t &= \boldsymbol{\pi}(\mathbf{x}_t) \end{aligned} \quad (2)$$

where $\boldsymbol{\sigma}_m$ and $\boldsymbol{\sigma}_a$ respectively denote the multiplicative and additive uncertainties between actual dynamics and its approximation, $\boldsymbol{\sigma}(\mathbf{x}_t)$ denotes the lumped model uncertainty, and $\mathbf{u}_t = \boldsymbol{\pi}(\mathbf{x}_t)$ denotes the control given by the controller $\boldsymbol{\pi}$ at the state $\mathbf{x}_t$.

The system state $\mathbf{x}_t$ is estimated from some estimators (e.g. SLAM) and is thus corrupted with noises and is given as

$$\bar{\mathbf{x}}_t = \mathbf{x}_t(\mathbf{1} + \boldsymbol{\epsilon}_m) + \boldsymbol{\epsilon}_a = \mathbf{x}_t + \boldsymbol{\epsilon}(\mathbf{x}_t) \quad (3)$$

where $\boldsymbol{\epsilon}_m$, $\boldsymbol{\epsilon}_a$, and $\boldsymbol{\epsilon}(\mathbf{x}_t)$ respectively denote the multiplicative, additive, and lumped uncertainties of the state estimates.

As a result, the state at time $t + 1$ predicted by the dynamics approximation $\bar{\mathbf{f}}$ is given as

$$\bar{\mathbf{x}}_{t+1} = \bar{\mathbf{f}}(\bar{\mathbf{x}}_t, \bar{\mathbf{u}}_t) \quad (4)$$

where $\bar{\mathbf{u}}_t = \boldsymbol{\pi}(\bar{\mathbf{x}}_t)$ denote the control evaluated at the state estimate $\bar{\mathbf{x}}_t$.

Problem 1: Given an approximated dynamics model $\bar{\mathbf{f}}(\mathbf{x}_t, \mathbf{u}_t)$, a state estimator that outputs the system estimates $\bar{\mathbf{x}}_t$, control saturations $\mathbf{u}_{\max}$ and $\mathbf{u}_{\min}$, and an attractive region $\mathcal{D}$, find a controller $\boldsymbol{\pi}(\bar{\mathbf{x}}_t)$ such that the system can robustly stabilize at the equilibrium point $\mathbf{x}^*$ under the uncertainties $\boldsymbol{\sigma}(\mathbf{x})$ in dynamics modeling and $\boldsymbol{\epsilon}(\mathbf{x})$ in state estimates.

The controller $\boldsymbol{\pi}$ is trained in the simulator built on the approximated dynamics model $\bar{\mathbf{f}}$ and is then applied to the actual system $\mathbf{f}$, bringing about the sim-to-real gap in dynamics models. Due to the adversarial relationship between the control saturations and uncertainties, some states in $\mathcal{D}$ may not satisfy the robust conditions under any control. The attractive region $\mathcal{D}$ should be investigated before the learning process.

A smooth dynamical system can be approximated with a parameterized neural network according to universal approximation theorem [41]. In addition, Bayesian neural network (BNN) has been widely applied in uncertainty quantification. For dynamic systems, it provides the uncertainty of states (e.g., Gaussian distribution). Therefore, the confidence interval of the predicted state can be viewed as the uncertainty bounds, or variational inference can be used to learn an approximate distribution, from which samples can be drawn to obtain the uncertainty bounds [42]. As an alternative, a neural network can be trained to directly output uncertainty bounds given the difference between predicted states and samples. Therefore, two assumptions are as follows.

Assumption 1: The uncertainty $\boldsymbol{\sigma}(\mathbf{x})$ of system dynamics and the uncertainty $\boldsymbol{\epsilon}(\mathbf{x})$ of state estimates are safely assumed bounded and their bounds are respectively denoted as $\Delta_{\sigma}(\mathbf{x})$ and $\Delta_{\epsilon}(\mathbf{x})$, such that

$$|\boldsymbol{\sigma}(\mathbf{x})| \leq \Delta_{\sigma}(\mathbf{x}), \quad |\boldsymbol{\epsilon}(\mathbf{x})| \leq \Delta_{\epsilon}(\mathbf{x}) \quad (5)$$

where $|\cdot|$ is the infinite norm.

Assumption 2: The approximated dynamics model $\bar{\mathbf{f}}$ and the uncertainty bounds $\Delta_{\sigma}(\mathbf{x})$ and $\Delta_{\epsilon}(\mathbf{x})$ are given in the form of the ReLU neural network.

ReLU networks allow flexible modeling of the system dynamics and the uncertainty bounds, making the proposed RGRL applicable to many complex systems. As pointed out in Section V, under Assumptions 1 and 2, the robustness conditions can be converted into a set of linear constraints and the states that violate these constraints can be found efficiently by mixed-integer linear programming (MILP).

IV. ROBUSTNESS CONDITIONS

The robustness conditions, i.e., the stability conditions under the worst-case scenario of uncertainties, are derived to regulate the training of the controller and the Lyapunov network. When there is no uncertainty, the system stability can be guaranteed by learning a coupled convex Lyapunov function $V(\mathbf{x}_t): \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ such that

$$V(\mathbf{x}^*) = 0 \quad (6)$$

$$V_{\max}(\mathbf{x}_t) > V(\mathbf{x}_t) > V_{\min}(\mathbf{x}_t) > 0, \quad \forall \mathbf{x}_t \in \mathcal{D}, \mathbf{x}_t \neq \mathbf{x}^* \quad (7)$$

$$V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) < 0, \forall \mathbf{x}_t \in \mathcal{D}, \mathbf{x}_t \neq \mathbf{x}^* \quad (8)$$

where $\mathcal{D}$ denotes the attractive region and $\mathbf{x}^*$ the equilibrium state, $V_{\max}(\mathbf{x}_t)$ and $V_{\min}(\mathbf{x}_t)$ belong to the $\mathcal{K}$ function set. The stability conditions (6) to (8) ensure that the system starting at any state in $\mathcal{D}$ converges to the equilibrium state $\mathbf{x}^*$.

A. Robustness Condition Under Uncertainty in Dynamics Modeling

Under the model uncertainty $\sigma(\mathbf{x})$ in dynamics, the state $\bar{\mathbf{x}}_{t+1}$ at time $t + 1$ can not be exactly determined by the control $\mathbf{u}_t$ and the current state $\mathbf{x}_t$. Therefore, to obtain guaranteed robustness, (8) has to be valid for each possible state $\bar{\mathbf{x}}_{t+1}$ at $t + 1$, such that

$$V(\bar{\mathbf{x}}_{t+1}) - V(\mathbf{x}_t) < 0, \forall \mathbf{x}_t \in \mathcal{D}, \mathbf{x}_t \neq \mathbf{x}^* \quad (9)$$

where

$$\bar{\mathbf{x}}_{t+1} = \bar{\mathbf{f}}(\mathbf{x}_t, \mathbf{u}_t) + \sigma(\mathbf{x}_t). \quad (10)$$

By performing a Taylor expansion of the piecewise linear Lyapunov function at $\mathbf{x}_{t+1}$

$$V(\mathbf{x}) = V(\mathbf{x}_{t+1}) + \mathcal{G}_x V(\mathbf{x}_{t+1})(\mathbf{x} - \mathbf{x}_{t+1}) \quad (11)$$

where the subgradient operator is denoted as $\mathcal{G}$. Subdifferential is the set of all subgradients of the function $V(\mathbf{x})$ at $\mathbf{x}$, then any chosen subgradient [43] in subdifferential at $\mathbf{x}_{t+1}$ is written as $\mathcal{G}_x V(\mathbf{x}_{t+1})$. After substituting $\bar{\mathbf{x}}_{t+1}$ into (11), the robustness condition (8) becomes

$$V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) < -\mathcal{G}_x V(\mathbf{x}_{t+1})(\bar{\mathbf{x}}_{t+1} - \mathbf{x}_{t+1}), \quad \forall \mathbf{x}_t \in \mathcal{D}, \mathbf{x}_{t+1} \neq \mathbf{x}^*. \quad (12)$$

Given the stochastic nature of $\bar{\mathbf{x}}_{t+1}$ and based on Assumption 1, a stronger and sufficient robustness condition is obtained as below

$$V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) < -|\mathcal{G}_x V(\mathbf{x}_{t+1})|\Delta_\sigma(\mathbf{x}_{t+1}). \quad (13)$$

The worst scenario of the uncertainty in the dynamics modeling is considered, and the Lyapunov value is enforced to decrease along the state trajectory.

Remark 1: By fixing the trainable parameters of the Lyapunov network, the subgradient $\mathcal{G}_x V(\mathbf{x})$ of the Lyapunov function $V(\mathbf{x})$ with respect to (w.r.t.) the state $\mathbf{x}$ is piecewise constant. In addition, the uncertainty bound $\Delta_\sigma(\mathbf{x})$ and Lyapunov function are piecewise linear in $\mathbf{x}$. Therefore, the robust condition (13) is piecewise linear in $\mathbf{x}$.

B. Robustness Condition Under Uncertainty in State Estimates

Contrary to the control input $\mathbf{u}_t = \pi(\mathbf{x}_t)$ obtained given the ground-truth state $\mathbf{x}_t$, the controller π is actually evaluated at the state estimate $\bar{\mathbf{x}}_t$ to determine the control input $\bar{\mathbf{u}}_t$. Therefore, the stability condition needs to account for the stochastic nature of the state estimates.

Considering the worst-case uncertainty in state estimates, the Lyapunov value at $\bar{\mathbf{x}}_{t+1}$ can be obtained by Taylor expansion as follows:

$$\begin{aligned} V(\bar{\mathbf{x}}_{t+1}) &= V[\mathbf{f}(\mathbf{x}_t, \pi(\bar{\mathbf{x}}_t))] \\ &= V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}_t))] \\ &\quad + \mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))]\Big|_{\mathbf{x}=\mathbf{x}_t} (\bar{\mathbf{x}}_t - \mathbf{x}_t) \end{aligned} \quad (14)$$

where the subgradient of the Lyapunov function V w.r.t. $\mathbf{x}_t$ is obtained through the chain rule and is as

$$\begin{aligned} \mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))]\Big|_{\mathbf{x}=\mathbf{x}_t} \\ = \mathcal{G}_x V(\mathbf{x})\Big|_{\mathbf{x}=\mathbf{x}_{t+1}} \mathcal{G}_u \mathbf{f}(\mathbf{x}, \mathbf{u})\Big|_{\mathbf{x}=\mathbf{x}_t, \mathbf{u}=\mathbf{u}_t} \mathcal{G}_x \pi(\mathbf{x})\Big|_{\mathbf{x}=\mathbf{x}_t} \end{aligned} \quad (15)$$

where $\mathbf{x}_{t+1} = \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t)$ and $\mathbf{u}_t = \pi(\mathbf{x}_t)$, the subgradients $\mathcal{G}_x V(\mathbf{x})$, $\mathcal{G}_u \mathbf{f}(\mathbf{x}, \mathbf{u})$, $\mathcal{G}_x \pi(\mathbf{x})$ are still any of the corresponding subdifferentials.

Then, the robustness condition under the uncertainty in the state estimates is given as

$$\begin{aligned} V(\bar{\mathbf{x}}_{t+1}) - V(\mathbf{x}_t) &< -\mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))]\Big|_{\mathbf{x}=\mathbf{x}_t}, \\ &\forall \mathbf{x}_t \in \mathcal{D}, \mathbf{x}_t \neq \mathbf{x}^*. \end{aligned} \quad (16)$$

Under Assumption 1, the robustness condition (8) becomes

$$V(\bar{\mathbf{x}}_{t+1}) - V(\mathbf{x}_t) < -\left|\mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))]\Big|_{\mathbf{x}=\mathbf{x}_t}\right| \Delta_\epsilon(\mathbf{x}_t). \quad (17)$$

Remark 2: The bounded subgradients $\mathcal{G}_x V(\mathbf{x})$, $\mathcal{G}_u \mathbf{f}(\mathbf{x}, \mathbf{u})$, and $\mathcal{G}_x \pi(\mathbf{x})$ are piecewise constant in $\mathbf{x}$, in $\mathbf{u}$, and in $\mathbf{x}$, respectively. Therefore, the robust condition (17) is piecewise linear in $\mathbf{x}$.

V. ROBUSTNESS-GUARANTEED RL

Similar to stability-guaranteed RL [32], a controller network $\pi(\mathbf{x})$ and a Lyapunov network $V(\mathbf{x})$ are trained simultaneously. Let α denote the trainable parameters of the Lyapunov network and β denote the trainable parameters of the controller network. The robustness conditions (7), (13) and (17) are utilized to design the following loss functions to regulate the training of the controller network and the Lyapunov network.

The loss function to ensure the semipositive definiteness of the Lyapunov network, i.e., the robustness condition (7), is given as

$$\mathcal{L}_p = \max_{\mathbf{x} \in \mathcal{D}} \{-V(\mathbf{x}), 0\}. \quad (18)$$

When only the uncertainty σ in dynamics modeling is considered, the Lyapunov values along the trajectory should strictly monotonically increase and it is ensured by satisfying the robustness condition (13). The corresponding loss function is designed as

$$\mathcal{L}_\sigma = \max_{\mathbf{x}_t \in \mathcal{D}} \left\{ \begin{aligned} &V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) \\ &+ |\mathcal{G}_x V(\mathbf{x}_{t+1})|\Delta_\sigma(\mathbf{x}_{t+1}), 0 \end{aligned} \right\}. \quad (19)$$

Algorithm 1: Robust-Guaranteed RL.

```

1: Initialize controller  $\pi(\mathbf{x})$  and Lyapunov function  $V(\mathbf{x})$ 
2: while iterations  $\leq$  Maximum do
3:   Find  $\mathbf{x}^c$  by maximizing  $\gamma(\theta)$  in (23) via Gurobi
4:   Compute the gradient  $\partial\gamma/\partial\theta$ 
5:   Update  $\theta = \theta - \kappa\partial\gamma/\partial\theta$ , where  $\kappa$  is the step size
6: end while

```

When only the uncertainty ϵ in state estimates is considered, the loss function from robustness condition (17) is given as

$$\mathcal{L}_\epsilon = \max_{\mathbf{x}_t \in \mathcal{D}} \left\{ \begin{array}{l} V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) \\ + \left| \mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))] \right|_{\mathbf{x}=\mathbf{x}_t} \left| \Delta_\epsilon(\mathbf{x}_t) \right|, 0 \end{array} \right\}. \quad (20)$$

When both the uncertainties σ and ϵ are considered, the robustness condition (8) becomes

$$V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) < - \left| \mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))] \right|_{\mathbf{x}=\mathbf{x}_t} \left| \Delta_\epsilon(\mathbf{x}_t) \right| - |\mathcal{G}_x V(\mathbf{x}_{t+1})| \Delta_\sigma(\mathbf{x}_{t+1}). \quad (21)$$

Then, the loss function corresponding to this robustness condition is given as

$$\mathcal{L}_{\sigma\epsilon} = \max_{\mathbf{x}_t \in \mathcal{D}} \left\{ \begin{array}{l} V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) \\ + \left| \mathcal{G}_x V[\mathbf{f}(\mathbf{x}_t, \pi(\mathbf{x}))] \right|_{\mathbf{x}=\mathbf{x}_t} \left| \Delta_\epsilon(\mathbf{x}_t) \right|, 0 \\ + |\mathcal{G}_x V(\mathbf{x}_{t+1})| \Delta_\sigma(\mathbf{x}_{t+1}) \end{array} \right\}. \quad (22)$$

Let $\theta \triangleq (\alpha, \beta)$ denote the set of all trainable parameters of the controller and Lyapunov networks. The proposed RGRL based on the above-mentioned losses is summarized in Algorithm 1. It iterates between finding the *counter-example* state with θ fixed (Step 3 in Algorithm 1) and updating θ with the gradient evaluated at the found *counter-example* (Step 5 in Algorithm 1).

Particularly, in each iteration of training, the evaluations of the losses (19), (20), or (22) involve solving the state $\mathbf{x}$ ($\mathbf{x}_t$, $\mathbf{x}_{t+1}$) that mostly violate the robustness conditions. This state is defined as the above-mentioned *counter-example* and is denoted as $\mathbf{x}^c$. According to Assumptions 1 and 2, the activation functions in the controller network, the Lyapunov network, and the networks of uncertainty bounds are “Leaky-ReLU”. Then, based on Remarks 1–2, as inspired by [32], [44], the losses (19), (20), and (22) are piecewise linear functions in $\mathbf{x}$. The trainable parameters θ from the previous iteration are fixed and used when solving for $\mathbf{x}^c$. The attractive region $\mathcal{D}$ is decomposed into multiple partitions, which are determined by θ . Let $\mathcal{V}$ denote the set of the states at the partition vertices.

Let y_i be a binary indicator that equals 1 if and only if $\mathbf{x}$ is in the i th partition. Then, $\mathbf{x}$ is expressed as a convex combination of the states in $\mathcal{V}$. By introducing slack variables, finding the *counter-example* $\mathbf{x}^c$ can be converted into a problem of MLIP as follows:

$$\gamma(\theta) = \max_s \mathbf{c}_\theta^T \mathbf{s} + \mathbf{d}_\theta$$

$$\text{s.t. } \mathbf{A}_\theta \mathbf{s} = \mathbf{b}_\theta \quad (23)$$

where the problem coefficients/bounds $\mathbf{c}_\theta$, $\mathbf{d}_\theta$, $\mathbf{A}_\theta$, and $\mathbf{b}_\theta$ are all explicit functions of θ , which is considered fixed in finding the *counter-example*. The variable vector $\mathbf{s}$ to be optimized consists of $\mathbf{x}$ and $y_1, \dots, y_{n_V}$, where n_V is the cardinality of $\mathcal{V}$.

The objective function of this optimization problem is to maximize a specific loss function, which can be any one of (19), (20), or (22). For example, (22) can be transformed into

$$\begin{aligned} \max \quad & a_{\text{ReLU}}^T \mathbf{s} + b_{\text{ReLU}} - V(\mathbf{x}_t) + \lambda \cdot (\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})) \\ \text{s.t.} \quad & \mathbf{A}_\theta \mathbf{s} = \mathbf{b}_\theta \end{aligned} \quad (24)$$

where a_{ReLU} and b_{ReLU} are the linearization coefficients of ReLU network, and λ is the weight factor. As a result, the robustness conditions are converted into a set of linear constraints, and the *counter-example* can be found efficiently by mix-integer linear programming (MILP). Then, the optimal value of the objective function (23) can be written in the closed form as $\gamma(\theta) = \mathbf{c}_\theta^T \mathbf{A}_\theta^{-1} \mathbf{b}_\theta + \mathbf{d}_\theta$. The gradient $\partial\gamma(\theta)/\partial\theta$ is used to optimize the trainable parameters θ via back-propagation (Step 5 in Algorithm 1). The iterations continue until all losses converge to zero.

In our approach, we directly utilize the original rewards returned by the environment to propagate through the Bellman equation when calculating the Q target value

$$Q^{\text{target}}(s, a) = r(s, a) + \gamma_q(1 - d) \min_{i=1,2} Q_{\theta'_i}(s', \pi_{\phi'}(s')) + \xi_q \quad (25)$$

where γ_q represents the discount factor, d is the termination flag, and ξ_q introduces general noise. The loss function for the Q network is defined as

$$\mathcal{L}_Q = \mathbb{E}_{(s,a)} \left[\sum_{i=1}^2 (Q_{\theta_i}(s, a) - Q^{\text{target}}(s, a))^2 \right]. \quad (26)$$

We leverage batch data from the replay buffer to formulate the Lyapunov loss function in RL form

$$\begin{aligned} \mathcal{L}_V = \mathbb{E} [& \text{ReLU}(-V(s')) \\ & + \lambda_1 \mathbb{E} [\text{ReLU}(\Delta V + \eta \|\nabla_s V(s)\| \delta)] + \lambda_2 \mathbb{E} [V(s_0)^2] \end{aligned} \quad (27)$$

where λ_1 and λ_2 adjust the strength of degressive constraints and the regularization term separately, and η adjusts the strength of gradient robustness. $\Delta V = V(s') - V(s)$, and δ denotes uncertainty boundary here.

To ensure the Lyapunov value decreases sufficiently after policy updates, we incorporate constraints into the actor policy's optimization objectives, balancing reward maximization and robustness

$$\mathcal{L}_\phi = \underbrace{-\mathbb{E}_s [Q_{\theta_1}(s, \pi_\phi(s))]}_{\text{Reward maximization}} + \lambda_b \underbrace{\mathbb{E} [\text{ReLU}(\Delta V + \eta \|\nabla V\| \delta)]}_{\text{Robustness penalty}} \quad (28)$$

where λ_b adjusts the weight between the optimization objectives, achieving a balance between reward and robustness.

The solution (θ^*) obtained from the RGRL is determined by the uncertainties, the system dynamics, the control saturation, the attractive region, and the flexibility of the neural networks. The solvability of Problem 1 depends on the MILP problem in (23) and the control saturation and is investigated in the next section.

Upon completing the training process, we acquire a network that embodies the Lyapunov function $V(x)$. The value of $V(x)$ at each step satisfies (21), indicating that $V(x)$ exhibits a strictly monotonic decreasing behavior at each step. Precisely, for any time t , the inequality $V(x_{t+1}) < V(x_t)$ holds true. Given that $V(x)$ has a lower bound (i.e., $V(x) \geq 0$) within $\mathcal{D}$, the convergence theorem for monotonic bounded sequences ensures that $V(x)$ must converge to a non-negative value V' . Mathematically, this is expressed as $\lim_{t \rightarrow \infty} V(x_t) = V'$.

The system state x is compelled to converge to a specific domain defined as

$$\mathcal{D}_{V'} = \{x \in \mathcal{D} | V(x) \leq V'\}. \quad (29)$$

Given that $V(x)$ monotonically decreases and converges to V' , for any $\epsilon > 0$, there exists $T > 0$ such that for all $t > T$, the inequality $|V(x_t) - V'| < \epsilon$ holds. This implies

$$V(x_t) \leq V' + \epsilon \quad (30)$$

and thus, x_t must enter the set $\mathcal{D}_{V'}$ as $t \rightarrow \infty$.

Within the set $\mathcal{D}_{V'}$, since $V(x) \leq V'$ and $\dot{V}(x) < 0$, the system state x continues to evolve such that $V(x)$ decreases until it reaches V' . Consequently, the system state x converges to a stable subset within $\mathcal{D}_{V'}$.

VI. SOLVABILITY ANALYSIS

Due to the adversarial relation between the uncertainty magnitudes and control saturation, there may be no such controller that can stabilize the *counter-example* states with guaranteed robustness. As a result, the attractive region $\mathcal{D}$ should be carefully examined before using RGRL to find the robust controller and the coupled Lyapunov function. This section investigates the solvability of the RL problem given the uncertainties and control saturation.

There always exists a PWL function that corresponds to a ReLU neural [45]. PWL function has been widely used in many fields, including control systems, signal processing, optimization problems, circuit design, etc. The rich analysis tools for the PWL system can be adopted. When no uncertainty or control saturation is considered, according to [32], it has been proved that if a PWL system is affine, then there exists a piecewise linear Lyapunov function and a controller that stabilizes the system. In addition, if the states at the partition vertices $\mathcal{V}$ satisfy the stability condition, then all states in the partition satisfy the stability condition. This section extends this line of work by considering the control saturation and the uncertainties.

A. High-Level PWL

To further formulate the problem (23) into a more compact form without resorting to slack variables, a high-level canonical

representation of PWL functions is adopted and is referred to as HL-PWL. The partitions of the state space are given in Appendix B. HL-PWL has been proven to use fewer parameters than classical PWL while maintaining the same accuracy. The number of parameters used by PWL is $(k/k+1)^n(n+1)!$ times that used by HL-PWL, here k denotes the number of subregions used for partitioning ([46] Section V). HL-PWL is in the following form:

$$F(x) = a^T \hat{\eta}(x) \quad (31)$$

where $a = [a_0^T, a_1^T, \dots, a_n^T]^T$ is a coefficient vector and $\hat{\eta}(x) = [(\hat{\eta}^0(x))^T, (\hat{\eta}^1(x))^T, \dots, (\hat{\eta}^n(x))^T]^T$ is an aggregation of stacks of the basis functions. A stack of the basis functions $\hat{\eta}^i(x)$ includes all the nesting functions of level i [see (64) in Appendix A]. The coefficient vector a_i is associated with the nesting function $\hat{\eta}^i(x)$.

B. Robustness Conditions in HL-PWL Representation

The positive definiteness of the Lyapunov function, i.e., (7), is rewritten as

$$\xi_1(|x|) \leq V(x) \leq \xi_2(|x|), \quad \forall x \neq x^* \in \mathcal{D} \quad (32)$$

where $\xi_1(|x|)$ and $\xi_2(|x|)$ are $\mathcal{K}$ functions, which are positive definite and increase as $|x|$ increases.

Both the Lyapunov function $V(x)$ and the $\mathcal{K}$ function are rewritten in the HL-PWL form (31) as

$$V(x) = a^T \hat{\eta}(x) \quad (33)$$

$$\xi_i(|x|) = b_i^T \mu(|x|), i = 1, 2 \quad (34)$$

where $|\cdot|$ is the infinite norm, $a \in \mathbb{R}^{\zeta_a}$ and $b_i \in \mathbb{R}^{\zeta_i}$ are parameter vectors and according to [47], and ζ_a and ζ_i are the number of parameters. The upper and lower bounds $\xi_0(|x|)$, $\xi_1(|x|) \in \text{PWL}_{|x|}[0, |x|_{\max}] \rightarrow \mathbb{R}^1$, and $\mu(|x|) = [\mu^0(|x|), \mu^1(|x|)]^T$ is a vector function including nesting functions of levels 0 and 1. The maximum value of $|x|$ in $\mathcal{D}$ can be easily obtained. Because μ is a $\mathcal{K}$ function, then

$$\xi_i(0) = 0 \quad (35)$$

$$\xi_i(|x_{t+1}|) > \xi_i(|x_t|), \text{ if } |x_{t+1}| > |x_t|. \quad (36)$$

Then, the robustness condition (6) at the equilibrium and the positiveness condition (7) can be respectively rewritten as

$$\begin{aligned} \hat{\eta}(x^*)^T a &= 0 \\ \mu^T(|x|) b_1 - \hat{\eta}^T(x) a &\leq 0. \end{aligned} \quad (37)$$

Given the attractive domain $\mathcal{D}$ and its partitions, the condition (37) is converted into the form $W_1^T [b_1^T, a^T]^T \leq 0$, where $W_1 = [W_{11}^T, W_{12}^T]^T$. The matrices W_{11} and W_{12} are obtained by stacking the vectors $\mu(|x_j|)$ and $-\hat{\eta}(x_j)$ for all x_j in $\mathcal{V}$, where $\mathcal{V}$ is the set of the vertices from the partitions of $\mathcal{D}$. Similarly, the upper bound condition in (32) can be reformed into $W_1^T [b_2^T, a^T]^T \geq 0$. The upper bound and lower bounds $\xi_1(|x|)$ and $\xi_2(|x|)$ are $\mathcal{K}$ functions that satisfying the condition (35), i.e.,

$$\mu^T(0) b_i = 0, i = 1, 2 \quad (38)$$

and the condition (36), i.e.,

$$\mathbf{W}_2^T \mathbf{b}_i \prec \mathbf{0}, i = 1, 2 \quad (39)$$

where $\mathbf{W}_2 = \{\boldsymbol{\mu}(0) - \boldsymbol{\mu}(\gamma), \dots, \boldsymbol{\mu}((m-1)\gamma) - \boldsymbol{\mu}(m\gamma)\}$ and γ is the grid size of the partitions. Also according to [46], for PWL systems, it is sufficient to analyze the state at the vertices $\mathcal{V}$. Notice that in the remainder discussion, without causing any ambiguity, the state $\mathbf{x}_t$ is assumed as one of the vertices in $\mathcal{V}$.

When both the uncertainties in dynamics modeling and state estimates are considered, the robustness conditions (21) and (15) yield another condition

$$\begin{aligned} & V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t) \\ & < -|\mathcal{G}_x V(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_{t+1}} |\mathcal{G}_u \mathbf{f}(\mathbf{x}, \mathbf{u})|_{\substack{\mathbf{x}=\mathbf{x}_t \\ \mathbf{u}=\mathbf{u}_t}} \\ & \quad \times |\mathcal{G}_x \boldsymbol{\pi}(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_t} \Delta_\epsilon(\mathbf{x}_t) - |\mathcal{G}_x V(\mathbf{x}_{t+1})| \Delta_\sigma(\mathbf{x}_{t+1}). \end{aligned} \quad (40)$$

It is further assumed that both the dynamics system and the controller satisfy the following Lipschitz conditions:

$$\left| \mathcal{G}_x \boldsymbol{\pi}(\mathbf{x}) \right|_{\mathbf{x}=\mathbf{x}_t} \leq \mathcal{P}(\mathbf{x}_t), \quad \left| \mathcal{G}_u \mathbf{f}(\mathbf{x}, \mathbf{u}) \right|_{\substack{\mathbf{x}=\mathbf{x}_t \\ \mathbf{u}=\mathbf{u}_t}} \leq \mathcal{F}(\mathbf{x}_t) \quad (41)$$

where $\mathcal{P}(\mathbf{x}_t)$ and $\mathcal{F}(\mathbf{x}_t)$ are positive and represent the upper bounds of the subgradients.

Then, the Taylor expansion of the term $V(\mathbf{x}_{t+1}) - V(\mathbf{x}_t)$ at the state $\mathbf{x}_{t+1}$ yields

$$\begin{aligned} & \mathcal{G}_x V(\mathbf{x}_{t+1})(\mathbf{x}_{t+1} - \mathbf{x}_t) \\ & < -|\mathcal{G}_x V(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_{t+1}} |\mathcal{G}_u \mathbf{f}(\mathbf{x}, \mathbf{u})|_{\substack{\mathbf{x}=\mathbf{x}_t \\ \mathbf{u}=\mathbf{u}_t}} \\ & \quad \times |\mathcal{G}_x \boldsymbol{\pi}(\mathbf{x})|_{\mathbf{x}=\mathbf{x}_t} \Delta_\epsilon(\mathbf{x}_t) - |\mathcal{G}_x V(\mathbf{x}_{t+1})| \Delta_\sigma(\mathbf{x}_{t+1}). \end{aligned} \quad (42)$$

By using the HL-PWL representation, (42) becomes

$$\begin{aligned} & \mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1})(\mathbf{x}_{t+1} - \mathbf{x}_t) \\ & < -|\mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1})| |\mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})| \end{aligned} \quad (43)$$

where $\nabla \hat{\eta}(\mathbf{x}_{t+1})$ denotes the derivatives of $\hat{\eta}(\mathbf{x}_{t+1})$ w.r.t. $\mathbf{x}_{t+1}$. Then, the robust condition (40) becomes

$$\begin{aligned} & \mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1})(\mathbf{x}_{t+1} - \mathbf{x}_t)(\mathbf{x}_{t+1} - \mathbf{x}_t)^T \nabla \hat{\eta}^T(\mathbf{x}_{t+1}) \mathbf{a} \\ & > -|\mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1})| |\mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})| \\ & \quad (\mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})) |\nabla \hat{\eta}^T(\mathbf{x}_{t+1}) \mathbf{a}|. \end{aligned} \quad (44)$$

It can be written as the following quadratic inequality:

$$\mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1}) \left\{ \begin{array}{l} (\mathbf{x}_{t+1} - \mathbf{x}_t)(\mathbf{x}_{t+1} - \mathbf{x}_t)^T \\ -|\mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})|^2 \mathbf{I} \end{array} \right\} \nabla \hat{\eta}^T(\mathbf{x}_{t+1}) \mathbf{a} > 0 \quad (45)$$

where $(\mathbf{x}_{t+1} - \mathbf{x}_t)$ is column vector, $\mathcal{X} \triangleq (\mathbf{x}_{t+1} - \mathbf{x}_t)(\mathbf{x}_{t+1} - \mathbf{x}_t)^T$ is a real symmetric matrix. Since $\mathcal{X}$ is a rank-one matrix, it has one non-zero eigenvalue, which equals the square of the norm of $(\mathbf{x}_{t+1} - \mathbf{x}_t)$, i.e., $|\mathbf{x}_{t+1} - \mathbf{x}_t|^2$. The rest of the eigenvalues are zero and let

$$\boldsymbol{\Lambda} \triangleq \begin{bmatrix} |\mathbf{x}_{t+1} - \mathbf{x}_t|^2 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}. \quad (46)$$

Then, $\mathcal{X}$ can be rewritten as

$$\mathcal{X} = \mathbf{Q} \boldsymbol{\Lambda} \mathbf{Q}^T \quad (47)$$

where $\mathbf{Q}$ is an orthogonal identity matrix ($\mathbf{Q}\mathbf{Q}^T = \mathbf{I}$). Let $\tau_{\mathbf{x}_{t+1}} \triangleq \mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})$. As a result, (45) can be further rewritten as

$$\mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1}) \mathbf{Q} [\boldsymbol{\Lambda} - \tau_{\mathbf{x}_{t+1}}^2 \mathbf{I}] \mathbf{Q}^T \nabla \hat{\eta}^T(\mathbf{x}_{t+1}) \mathbf{a} > 0 \quad (48)$$

where

$$\begin{aligned} & \boldsymbol{\Lambda} - \tau_{\mathbf{x}_{t+1}}^2 \mathbf{I} \\ & = \begin{bmatrix} |(\mathbf{x}_{t+1} - \mathbf{x}_t)|^2 - \tau_{\mathbf{x}_{t+1}}^2 & 0 & \cdots & 0 \\ 0 & -\tau_{\mathbf{x}_{t+1}}^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & -\sigma_{\mathbf{x}_{t+1}}^2 \end{bmatrix}. \end{aligned} \quad (49)$$

It shows that $\mathbf{a}^T \nabla \hat{\eta}(\mathbf{x}_{t+1}) \mathbf{Q} [\boldsymbol{\Lambda} - \tau_{\mathbf{x}_{t+1}}^2 \mathbf{I}] \mathbf{Q}^T \nabla \hat{\eta}^T(\mathbf{x}_{t+1}) \mathbf{a}$ can be converted into a standard quadratic form and $|\mathbf{x}_{t+1} - \mathbf{x}_t|^2 - \tau_{\mathbf{x}_{t+1}}^2$ has to be positive to the existence of a solution.

However, the controls are saturated at the lower bound $\mathbf{u}_{\min}$ and the upper bound $\mathbf{u}_{\max}$. If for all $\mathbf{u}_{\min} \preceq \mathbf{u}_t \preceq \mathbf{u}_{\max}$, $|\mathbf{x}_{t+1} - \mathbf{x}_t| < \mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1})$, then regardless of the variations in coefficients $\mathbf{a}$, the entire quadratic form (48) remains less than zero and the robust condition (40) can not be valid. Therefore, the necessary condition for the existence of the solution to the RL problem, i.e., existing $V(\mathbf{x})$ and $\boldsymbol{\pi}(\mathbf{x})$ that satisfy the robust conditions (40), is given as

$$|\mathbf{x}_{t+1} - \mathbf{x}_t| > \mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{x}_{t+1}). \quad (50)$$

This necessary condition (50) requires that the controller possesses adequate capability to bring the system to $\mathbf{x}_{t+1}$ from $\mathbf{x}_t$, such that $|\mathbf{x}_{t+1} - \mathbf{x}_t|$ can transcend the uncertainty in states. For each state $\mathbf{x}_t$ in $\mathcal{V}$, all possible controls that satisfy the necessary condition constitute an admissible control set $\tilde{\mathcal{U}}$. For any control $\tilde{\mathbf{u}}$ in $\tilde{\mathcal{U}}$, there exists a subset $\tilde{\mathcal{A}}$ in the space of $\mathbf{a}$, such that any $\mathbf{a}$ in $\tilde{\mathcal{A}}$ satisfies the robust condition (45). Then, for each vertex state $\mathbf{x}_i \in \mathcal{V}$, the solution space $\mathcal{A}$ is the union of all $\tilde{\mathcal{A}}$ for all admissible control in $\tilde{\mathcal{U}}$. Let $\mathcal{H}$ denote the boundary of $\tilde{\mathcal{U}}$. Since the system (2) is PWL and $\tilde{\mathcal{U}}$ is convex, it is sufficient to obtain $\mathcal{A}$ by evaluating the control in $\mathcal{H}$, which is approximated by a pair of symmetric cones $\mathcal{C}_i$.

Consequently, the solution space of $\mathbf{a}$ regarding each state $\mathbf{x}_i$ in $\mathcal{V}$ corresponds to a pair of symmetric cones $\mathcal{C}^i$, and the intersection $\mathcal{A}^*$ of these cones $\mathcal{C}_i$ is the solution space of $\mathbf{a}$. As an example, in Fig. 2, the cones corresponding to four vertex states are considered. For pair cones are aligned and considering the constraints imposed by (37), only four cones are displayed and all other symmetric cones are neglected. The intersection of these four cones can be approximated by linear constraints, illustrated by red dots in Fig. 2, and it is the solution space $\mathcal{A}^*$ of $\mathbf{a}$.

Without loss of generality, assume that the cones have N intersections $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_N$ (illustrated as red dots in Fig. 2). These intersections form N planes in the solution space. The normal

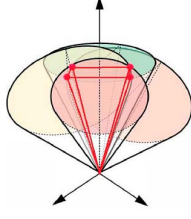


Fig. 2. Solution space constituted by cones.

vector $\mathbf{n}_1, \mathbf{n}_2, \dots, \mathbf{n}_N$ is calculated for each plane to represent the linear cone formed by these N intersections. Therefore

$$\mathbf{n}_i^T \mathbf{a} > 0, \quad \mathbf{n}_i = \mathbf{v}_i \times \mathbf{v}_{i+1} \quad (51)$$

is the linear constraints on $\mathbf{a}$.

Then the solution to the parameters of the Lyapunov candidate function $V \in \text{PWL}[S] : S \rightarrow \mathbb{R}^1$ is constrained by

$$\left. \begin{aligned} \hat{\eta}(\mathbf{x}^*)^T \mathbf{a} &= 0, \mathbf{W}_1^T \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{a} \end{bmatrix} \leq 0, \mathbf{W}_1^T \begin{bmatrix} \mathbf{b}_2 \\ \mathbf{a} \end{bmatrix} \geq 0 \\ \mathbf{n}_i^T \cdot \mathbf{a} &\geq 0, \mu^T(\mathbf{x}^*) \mathbf{b}_i = 0, \mathbf{W}_2^T \mathbf{b}_i < 0, i = 1, 2 \end{aligned} \right\} \quad (52)$$

where $\mathbf{b}_1, \mathbf{b}_2, \mathbf{a}$ are the coefficients of the HL-PWL representation of the Lyapunov function and its bounds.

By using the HL-PWL representation, the robustness conditions are then shown linear in $\mathbf{a}$, $\mathbf{b}_1$, and $\mathbf{b}_2$. Notice that the control parameters are not shown in the equation set (52). The existence of a Lyapunov function indicates the existence of a controller since the control saturation has been fully considered in the constraints (51). Therefore, the exact solution set can be obtained by finding the set constrained by the equations in (52) to show the solvability of the RL problem.

C. Prevalidation of Attractive Region

In the existing literature, the attractive region $\mathcal{D}$ is often given without careful investigation. As a result, some vertices in $\mathcal{V}$ may not satisfy the necessary condition (50), making Problem 1 unsolvable. This necessary condition requires that the controller possesses adequate capability against uncertainties. By using this necessary condition, the infeasible state can be found, and the attractive region $\mathcal{D}$ can be improved by eliminating the states that cannot be guaranteed.

Given the control upper and lower bounds $\mathbf{u}_{\min}$ $\mathbf{u}_{\max}$ and uncertainty bounds $\Delta_\epsilon(\mathbf{x})$ and $\Delta_\sigma(\mathbf{x})$, all the states at the partition vertices $\mathcal{V}$ are checked against the necessary condition (50). The ones that satisfy (50) constitute a new vertex set $\mathcal{V}'$. Then a refined attractive region $\mathcal{D}'$ is obtained. According to [46], if the vertices in $\mathcal{V}'$ satisfy the necessary condition (50), then all states in $\mathcal{D}'$ satisfy that condition.

D. Maximum Robustness

Proposition 1: The controller becomes more robust if the uncertainty bounds $\Delta_\epsilon(\mathbf{x})$ and $\Delta_\sigma(\mathbf{x})$ are amplified by $\rho < \rho_{\max}$

during training in using RGRL, where $\rho_{\max}$ is the maximal value and is given as

$$\rho_{\max} = \min_{\mathbf{x}_t \in \mathcal{D}', \mathbf{u}_{\min} \preceq \mathbf{u} \preceq \mathbf{u}_{\max}} \frac{|\mathbf{f}(\mathbf{x}_t, \mathbf{u}) - \mathbf{x}_t|}{\mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{f}(\mathbf{x}_t, \mathbf{u}))}. \quad (53)$$

Any $\rho > \rho_{\max}$ makes the RL problem unsolvable.

Proof: The first diagonal term in the necessary condition (50) is positive and other terms are negative. The quadratic form defines a second-type cone, also referred to as a hyperbolic cone. The cone angle can be approximated by considering the angle between the cone and its principal axis. In addition, the principal axis of the cone aligns with the direction of the eigenvector corresponding to this positive eigenvalue.

Then, the half-angle θ of the cone can be approximated by

$$\theta = \arctan \left(\sqrt{\frac{\|(\mathbf{x}_{t+1}^p - \mathbf{x}_t^p)\|^2 - \sigma_{\mathbf{x}_{t+1}}^2}{\sigma_{\mathbf{x}_{t+1}}^2}} \right). \quad (54)$$

Assuming that the uncertainty of the system increases to $\rho\sigma_{\mathbf{x}_{t+1}}^2$ and the necessary condition becomes $|\mathbf{x}_{t+1}^p - \mathbf{x}_t^p| > \rho|\sigma|$. The new half-angle θ' becomes

$$\theta' = \arctan \left(\sqrt{\frac{\|(\mathbf{x}_{t+1}^p - \mathbf{x}_t^p)\|^2 - \rho^2\sigma_{\mathbf{x}_{t+1}}^2}{\rho^2\sigma_{\mathbf{x}_{t+1}}^2}} \right). \quad (55)$$

Since the arctangent function is monotonically increasing, when the uncertainty of the system is amplified by the proportional coefficient ρ , the cone angle of the quadratic form decreases. Furthermore, the size of the cone angle is equivalent to the size of the solution space $\mathcal{A}^*$. Therefore

$$\tau_{\mathbf{x}_{t+1}} \longrightarrow \rho\tau_{\mathbf{x}_{t+1}}, \quad \mathcal{A}^* \longrightarrow \mathcal{A}'^* \quad (56)$$

where $\mathcal{A}^*$ and $\mathcal{A}'^*$ denote the solution sets of $\mathbf{a}$, respectively. As the cone angle decreases, the solution set is reduced to $\mathcal{A}'$, making the proposed RGRL method challenging but resulting in a controller that can deal with larger uncertainties.

The maximum value of this coefficient ρ is then obtained necessity condition (50). For all $\mathbf{x}_t \in \mathcal{V}'$, $\mathbf{u}_{\min} \preceq \mathbf{u} \preceq \mathbf{u}_{\max}$, the upper bound $\rho_{\max}$ of the amplification coefficient should satisfy the following condition:

$$\{\mathcal{P}\mathcal{F}\Delta_\epsilon(\mathbf{x}_t) + \Delta_\sigma(\mathbf{f}(\mathbf{x}_t, \mathbf{u}))\} \rho_{\max} \leq |\mathbf{f}(\mathbf{x}_t, \mathbf{u}) - \mathbf{x}_t|. \quad (57)$$

■

VII. SIMULATION RESULTS

In this section, we conducted simulations on an inverted pendulum and quadrotors. The controller and Lyapunov function were synthesized using the proposed RGRL and the obtained controller is referred to as the robust neural controller (RNC). RNC was compared against Lyapunov neural network control (NNC) and the robust iterative LQR (RiLQR) method. Both multiplicative and additive uncertainties, as defined in (3), were introduced into the dynamics modeling and state estimates. Specifically, we compared RGRL with domain randomization RL (DRRL) in terms of learning-based approach in pendulum,

TABLE I
PERFORMANCE OF METRICS BY λ_b

λ_b	0.1	0.5	1	2	4
θ error	0.1691	0.0419	0.2818	0.0127	0.2967
$\dot{\theta}$ error	0.0061	0.0052	0.0023	0.0021	0.0020
Lyapunov value	0.0016	0.0053	0.0057	0.0013	0.0794
Final reward	-0.0329	-0.0018	-0.0814	-0.0019	-0.0902
Converge time	0.2875	0.2771	0.31053	0.2292	0.2458

Note: Bold items are relatively optimal.

analyzed the distinct characteristics of the controller in high-dimensional systems, and explained boundaries adjustment in 3-D quadrotor. The physical parameters and neural network sizes are detailed in Table III in Appendix C.

A. Inverted Pendulum

The simulated inverted pendulum was built in Gym and the applied torque control ranged from -100 Nm to 100 Nm. Let θ denote the angle between the pendulum and the horizontal x-axis. The equilibrium point was set as $\theta = \pi$, $\dot{\theta} = 0$. The attractive region $\mathcal{D}$ of the state space $(\theta, \dot{\theta})$ was initially specified as $[0, 2\pi] \times [0, 4]$. Three scenes were simulated to illustrate the effectiveness of RNC in dealing with various uncertainties. In Scene I, only ϵ_m was considered. In Scene II, both ϵ_m and σ_m were considered. Scene III considered all kinds of uncertainties σ_m , σ_a , ϵ_m , and ϵ_a . The proposed method was also compared with DRRL. Before that, we analyzed the selection of an important parameter λ_b .

Weight Coefficient Analysis: This section evaluates the controller's performance under identical uncertainties for varying values of λ_b . As indicated in Table I, λ_b exerts a significant influence on the controller's effectiveness. The controller achieves its most favorable comprehensive performance at $\lambda_b = 2$. It should be noted that, due to the discrete nature of the tested λ_b values, the actual optimum may lie in the vicinity of 2.

1) Scene I - Multiplicative Uncertainty in State Estimates: In Scene I, the multiplicative uncertainty ϵ_m in state estimates was set to $[0.01 \ 0.01]^T$. Both the training of NNC and RNC converged within 5 and 3 h, respectively.

The controllers were tested with 30 different initial states. The resulting trajectories are shown in various colors in Fig. 3. The red star indicates the equilibrium point. NNC can only stabilize to the blue star state in the presence of ϵ_m [Fig. 3(a)], while the trajectories of RNC have successfully converged to the desired point [Fig. 3(d)].

As the evolution of the Lyapunov function over time depicted in Fig. 3(b) and (e), the Lyapunov values generated by RNC exhibited a consistent monotonic decrease, eventually reaching zero. In contrast, the Lyapunov values generated by NNC increased before converging to a non-zero constant value. Similar to the Lyapunov values, the control signals from NNC converged towards a non-zero value, while those from RNC converged towards zero as shown in Fig. 3(c) and (f).

The proposed RNC rigorously regulates the training process of the controller by robustness loss $\mathcal{L}_e$ (20). The robustness assurance provided by RNC can be assessed through the evaluation of value functions built on robustness constraints.

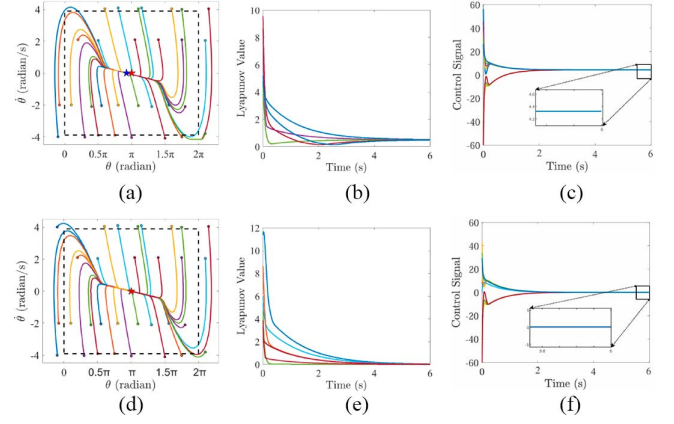


Fig. 3. Comparison of NNC and RNC under observation uncertainties: (a) trajectories NNC; (b) Lyapunov value NNC; (c) control signal NNC; (d) trajectories RNC; (e) Lyapunov value RNC; and (f) control signal RNC.

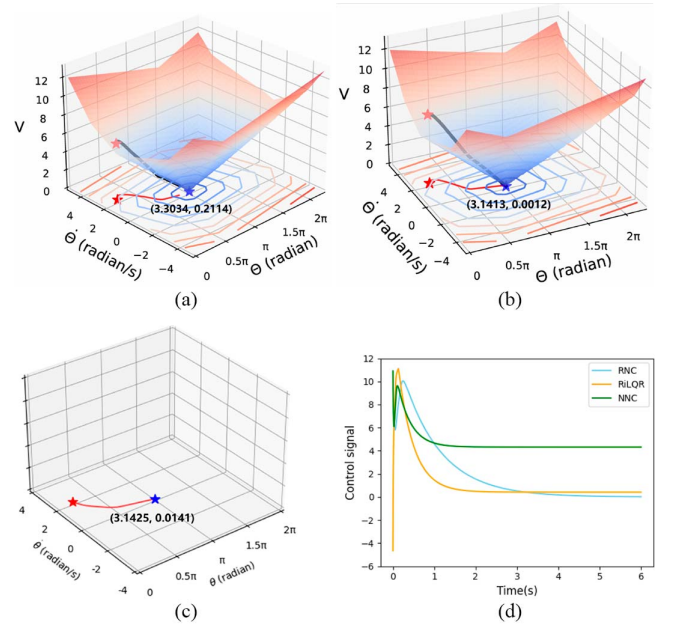


Fig. 4. Comparison of NNC, RNC and RiLQR controllers under observation uncertainty: (a) NNC; (b) RNC; (c) RiLQR; and (d) control signal.

The trajectories generated by NNC, RNC, and RiLQR are presented in Fig. 4. The Lyapunov functions for NNC and RNC are visualized as 3-D surfaces, with iso-value contours projected onto the bottom plane and centered around the equilibrium point. A red star denotes the initial state, and a blue star signifies the converged state. The system state evolved on the Lyapunov surface, under conditions of uncertainty in state estimates, the NNC struggled to reach the desired equilibrium state, whereas both RNC and RiLQR succeeded, with RNC achieving slightly higher control accuracy as shown in Fig. 4(d).

2) Scene II - Multiplicative Uncertainties in Dynamics Modeling and State Estimates: The uncertainty σ_m of the dynamics modeling was set to $[0.01 \ 0.01]^T$ and the uncertainty ϵ_m in the state estimates was set to $[0.01 \ 0.01]^T$. As shown in Fig. 5(a), NNC struggled to handle uncertainties in both the dynamics model and state estimates, causing the trajectory

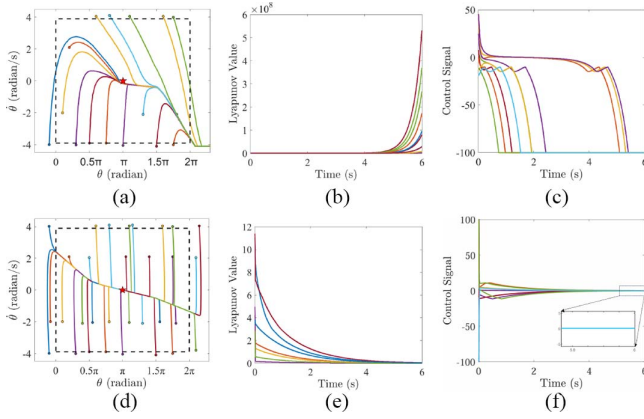


Fig. 5. Comparison of NNC and RNC controllers under model and observation uncertainties in multiplicative form: (a) trajectories NNC; (b) Lyapunov value NNC; (c) Control signal NNC; (d) trajectories RNC; (e) Lyapunov value RNC; and (f) control signal RNC.

to gradually deviate from the desired equilibrium state. As a consequence, the inverted pendulum continued to rotate without achieving a stationary state. This perpetual motion causes a continuous increase in the overall energy of the controlled system, resulting in control saturation and the exponential growth of the Lyapunov value, as shown in Fig. 5(b) and (c), respectively.

In contrast, RNC adeptly mitigated the influence of both uncertainties, effectively stabilizing the control system and guiding it toward the desired equilibrium state. The Lyapunov value decreased to zero, as can be seen in Fig. 5(e). In Fig. 5(d), all the trajectories quickly converged to a diagonal trajectory. The trajectories starting at initial states beyond $[0.5\pi, 1.5\pi]$ detoured a bit and then converged to the diagonal trajectory. It is because the multiplicative uncertainty required larger control inputs when the state was far away from the equilibrium point, as evident in Fig. 5(f). At the beginning of the trajectories, the control was often saturated, leading to rapid convergence to the diagonal trajectory. The guaranteed robustness is validated by the monotonical decrease in the Lyapunov value in Fig. 5(e).

As illustrated in Fig. 6 and detailed results in Table II, the example trajectories from both RNC and RiLQR converged successfully. In contrast, the NNC failed with control input saturation, indicating its limitations in handling uncertainties. The RNC demonstrates superior robustness and stability with a smoothly decreasing control signal and smaller control error. Conversely, RiLQR employs a more conservative control strategy, resulting in smaller fluctuations but slightly inferior robustness.

3) Scene III - Multiplicative and Additive Uncertainties in Modeling and State Estimates: In addition to the multiplicative uncertainties, Scene III also considered the additive uncertainties in the dynamics modeling and state estimates. The additive uncertainties ϵ_a and σ_a were set as $[0.01 \ 0.01]^T$ and $[0.01 \ 0.01]^T$.

In each row of Fig. 7, the left and right images illustrate the performances of the controllers in cases with and without additive uncertainties in state estimates σ_a . When σ_a was not considered, the obtained trajectories smoothly converged to the

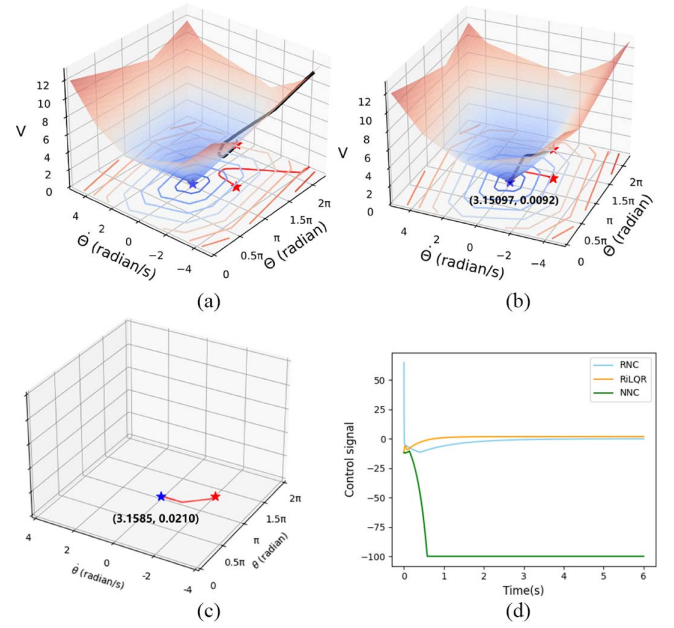


Fig. 6. Comparison of NNC, RNC, and RiLQR under model and observation uncertainties in multiplicative form: (a) NNC; (b) RNC; (c) RiLQR; and (d) control signal.

equilibrium state, as confirmed by the Lyapunov value and control signal. When σ_a was considered, as the trajectory was approaching the equilibrium state, oscillations in states were observed rather than stable convergence. Moreover, the Lyapunov value exhibited fluctuations near the equilibrium state, indicating that the controller can only ensure stable convergence beyond a certain distance to the equilibrium state.

4) Comparison Under the Same Uncertainties: In inverted pendulum simulations, we compared RGRL with DRRL from the perspective of learning-based methods. DRRL randomizes system parameters during training, showing some robustness to parameter variations, but lacks strict robustness guarantees. The training processes for RGRL and DRRL controllers converged within 5 and 3 h, respectively. The learning rate was set to 0.00003, and the coefficients were set as follows: $\lambda_1 = 1.5$, $\lambda_2 = 1.2$, and $\eta = 0.015$. A total of 100 independent test episodes were randomly generated, each consisting of 400 sequential steps. The uncertainties in each episode were randomly generated within predefined bounds and kept consistent across DRRL and RGRL tests to ensure fair comparison.

We performed a comprehensive statistical analysis across multiple test sets to evaluate the performance of RGRL and DRRL. As shown in Fig. 8, RGRL demonstrated significantly lower variance and effectively mitigated the impact of system uncertainty. Notably, at the 20-s mark, RGRL achieved an 88.6% reduction in average angular error compared with DRRL, underscoring its superior error suppression capability.

The average converged angular error for DRRL was 0.2215, whereas RGRL achieved an average error of 0.1270, representing a 57.3% reduction in angular error. This significant improvement can be attributed to RGRL's ability to effectively compensate for uncertainties, particularly in scenarios where

TABLE II
PERFORMANCE EVALUATION OF CONTROLLERS ACROSS MULTIPLE SCENARIOS

Metrics ¹	Pendulum (Scene II)			2-D Quadrotor (Scene V)			3-D Quadrotor (Hover)		
	RiLQR	NNC	RNC	RiLQR	NNC	RNC	RiLQR	NNC	RNC
Run Time (s)	8.01	22.4	18.2	53.0	79.3	75.6	-	-	-
Stable Pose Error	1.17E-2	Div.	9.5E-3	3.8E-3	Div.	3.8E-7	2.35E-2	3.42E-3	1.26E-4
Final Lyapunov	-	>1E3	1.3E-2	-	>1E3	9.5E-6	-	3.24E-4	1.22E-5
Control Error	1.62E-3	Sat.	1.4E-5	1.38E-5	5.62	2.5E-5	3.26E-3	1.47E-4	1.89E-5
Metrics ²	3-D Quadrotor (Trajectory tracking)								
	RMSE	MAE	Steady Error	Avg _{vel}	RMSE _{vel}	T _c (s)	E _u	Variance _u	Smoothness
RiLQR	0.12±0.02	0.09±0.02	0.11±0.018	0.54±0.15	0.31±0.04	2.8±1.0	150±30	0.276±0.04	0.78±0.03
RNC	0.08±0.015	0.06±0.035	0.05±0.032	0.51±0.08	0.27±0.08	3.4±0.8	187±20	0.018±0.03	0.85±0.02
2-D Quadrotor (Graded uncertainties ϵ, σ)			MILP Information			Pendulum	2-D Quadrotor	3-D Quadrotor	
Success Rate ³ (%)	Level-1	Level-2	Level-3	Model Size (rows, columns)			(380,162)	(496,210)	(816,328)
RiLQR	88±4	60±5	23±4	Presolved Model Size ⁴			(138,86)	(420,202)	(504,274)
RNC	98±1	87±2	72±4	Time Cost (s)			0.05	13.29	72.34

Note: ¹3-D quadrotor's orientation error: RiLQR(5.73E-4,-6.61E-4,3.38E-4), NNC(-1.99E-5,2.21E-5,7.54E-5), RNC(1.30E-6,3.38E-6,2.58E-6). Div: The trajectory has completely deviated from the equilibrium state. Sat: The control input is saturated. ²Median ± Variance (10 runs). T_c: Convergence time, E_u: Total Energy $\int_0^T c||u(t)||^2 dt$, Avg_{vel}: Average velocity. ³Mean ± SE. Standard error (SE) : $\sqrt{p \times (1-p)/n}$. 100 Monte Carlo episodes were conducted for each level. ⁴The pre-solution provided by Gurobi improves the efficiency of solving by removing redundant constraints and other methods. Time cost refers to the average solution time of MILP in one solution, and the cost increases significantly with the rise in system dimension. Bold entries are optimal.

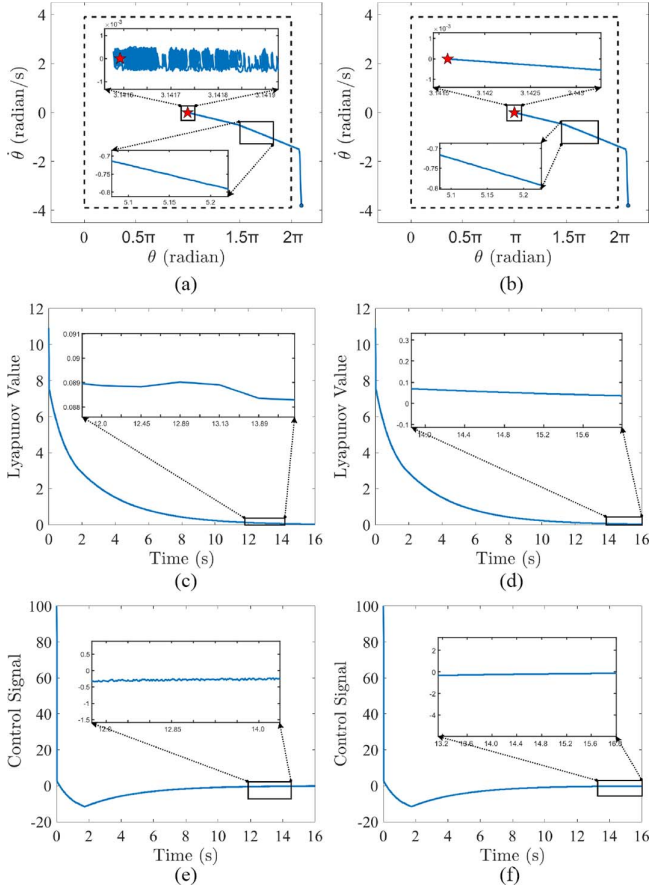


Fig. 7. Comparison of multiplicative and additive form uncertainties: (a) trajectories with σ_a ; (b) trajectories without σ_a ; (c) Lyapunov value with σ_a ; (d) Lyapunov value without σ_a ; (e) control signal with σ_a ; and (f) control signal without σ_a .

the control signal and uncertainty near equilibrium share similar magnitudes. Conventional RL approaches often struggle to capture system dynamics in such cases without

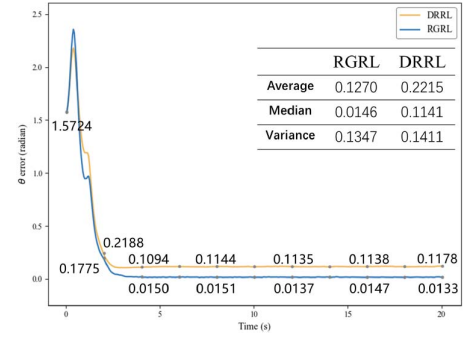


Fig. 8. Comparison of angular errors of DRRL and RGRL. Tracking the angle errors of the two methods at each step, and the results show that RGRL is more effective for handling uncertainties.

explicit robustness constraints, resulting in larger errors. RGRL demonstrates superior robustness and error control. This makes RGRL better suited for systems with inherent uncertainties.

B. Refinement of the Attractive Region

In this example, the uncertainty ϵ in dynamics modeling was set to $[0.04 \ 0.04]^T$ and the one in state estimates as $[0.4 \ 0.4]^T$. Before applying the proposed RGRL, the attractive region was refined. The new attractive region is illustrated in Fig. 9(a), where the states in the blue area could not satisfy the necessary condition and were subsequently excluded. The refined attractive region is depicted as the dashed red box. Then, a robust controller was successfully learned.

C. Learning Lyapunov Function

The learning process of the Lyapunov Function is visualized here. In RGRL, the Lyapunov function is trained based on the losses in (18), (19), or (20). The maximization operation in the bilevel optimization is achieved by solving the MILP problem.

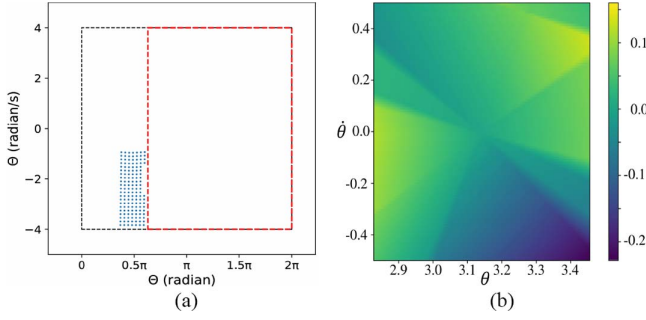


Fig. 9. Attractive region and robustness loss: (a) refining $\mathcal{D}$; and (b) robustness loss.

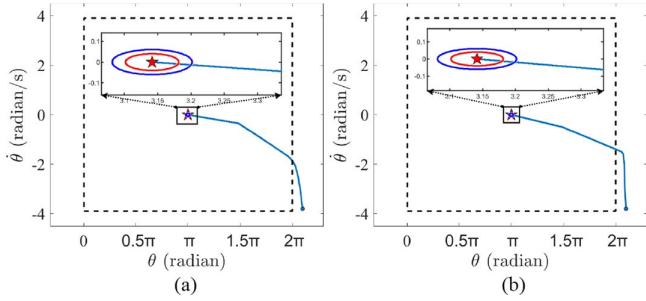


Fig. 10. Comparison of RNC and boosted RNC under the observation uncertainties: (a) RNC; and (b) boosted RNC.

The degrees of the states violating the robustness condition are analyzed. In Fig. 9(b), a snapshot of the losses over the entire state space was depicted after 100 training epochs. Initially, the state space appeared predominantly in yellow-green, indicating the violation of the robustness conditions given the initial controller network and the Lyapunov network. Initially, the *counter-examples* were the ones far away from the equilibrium states. It appears that the *counter-examples* gradually converged toward the equilibrium state as the training progressed. This is supported by the examples in Fig. 9(b), where the peripheral states were prioritized to be forced to satisfy the robustness condition during training.

D. Boosting Robustness Capability

During the training process, the uncertainties were purposely amplified by a factor of $\rho = 1.1$, resulting in a second RNC (referred to as boosted RNC). In Fig. 10, the blue circles represent the state where the Lyapunov value increased when the regular RNC was used, while the red circles represent the states where the Lyapunov values increased when the boosted RNC was used. It is observed that the boosted RNC performs better than the regular one, resulting in a smaller region of instability.

E. 2-D quadrotor

In this section, a 2-D quadrotor was simulated. The attractive region was designed as $[-1.5, -1, -0.5\pi, -4, -4, -2.75] \preceq [x, z, \theta, \dot{x}, \dot{z}, \dot{\theta}] \preceq [1.5, 1, 0.5\pi, 4, 4, 2.75]$. Controls are the thrusts of two motors $M1$ and $M2$. The equilibrium state

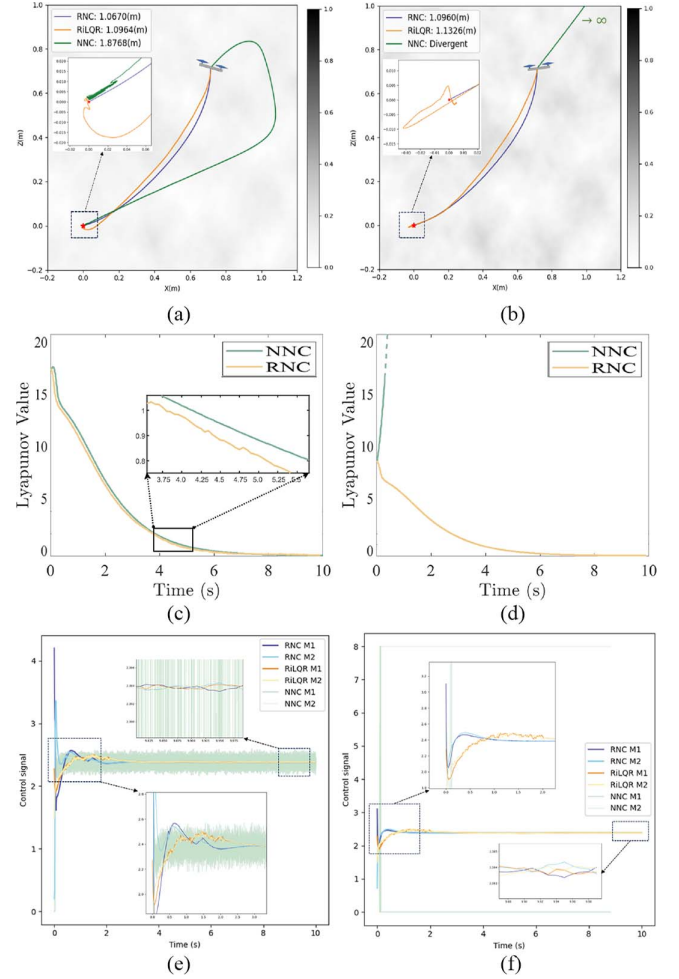


Fig. 11. Comparison of 2-D quadrotor performance under single (left) and combined uncertainties (right). (a) Trajectories with σ ; (b) trajectories with σ, ϵ ; (c) Lyapunov value with σ ; (d) Lyapunov value with σ, ϵ ; (e) control signal with σ ; and (f) control signal with σ, ϵ .

is origin point. Two scenes were simulated: Scene IV only included uncertainty in the dynamics modeling that σ was set as $[0.04, 0.04, 0, 0, 0, 0]^T$. Scene V included both uncertainties, where ϵ and σ were set as $[0.02, 0.02, 0, 0, 0, 0]^T$. Uncertainty is visualized through graded shades of gray: darker tones indicate higher uncertainty, lighter ones lower. The training of NNC and RNC converged within 6 and 12 h, respectively.

1) *Scene IV - Uncertainty in Dynamics Modeling:* The trajectories generated by RNC, RiLQR, and NNC are presented in Fig. 11(a). In this scenario, all controllers successfully steered the quadrotor toward the desired equilibrium point. Notably, the RNC achieved convergence (indicated in red) along a shorter path, without exhibiting chattering near the equilibrium point as observed with RiLQR, and NNC performed poorly due to the lack of rewards. The corresponding Lyapunov function value for the NNC exhibited intermittent increases, indicating a loss of guaranteed robustness. Conversely, the Lyapunov value for the RNC decreased monotonically and converged to zero after 10 s. The control signals, summarized in Fig. 11(e), show that NNC output exhibited frequent fluctuations, while RNC provides smooth signals with ensured robustness.

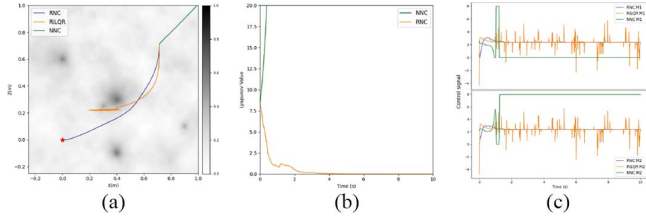


Fig. 12. Example of level-2. RiLQR exhibited severe oscillations, while RNC achieved stability and reached the equilibrium point: (a) trajectories; (b) lyapunov value; and (c) control signal.

2) *Scene V - Uncertainties in Dynamics Modeling and State Estimates*: The trajectories generated by RNC, RiLQR, and NNC are shown in Fig. 11(b). When controlled by RNC, the quadrotor stably converged to the equilibrium state (indicated in red). In contrast, the quadrotor failed to stabilize under NNC's control. The model-based RiLQR exhibited minor oscillations near the equilibrium point, resulting in deviations in the system response under varying noise conditions. This performance difference is further reflected in the Lyapunov function values along the trajectories. The Lyapunov value for NNC increased from the very beginning, while for RNC monotonically decreased and converged to zero after 10 s. The control signals are summarized in Fig. 11(f). NNC failed to stabilize the quadrotor, and its control was constantly saturated after 2 s. RNC output initially saturated and became smoother than RiLQR, eventually decreasing to constant values once the quadrotor stabilized.

3) *Tests with Three Levels of Uncertainties*: To evaluate the controller's robustness in practical settings, we introduce three levels of non-homogeneous uncertainty across the state domain. Using Scene V as a baseline, we scale the magnitudes of ϵ and σ by $2\times$, $4\times$, and $6\times$ to define Level-1, Level-2, and Level-3, respectively.

The success rates of three levels are shown in Table II. Subject to uncertainties of Level-2, RiLQR failed in 40% of episodes. This is primarily because partial state spaces, especially high-uncertainty subregions, do not satisfy solvability condition (50) as local uncertainty intensities exceed critical thresholds. These regions exacerbate the accumulation of RiLQR's linearization errors, ultimately causing system state divergence, as shown in Fig. 12. In contrast, RNC, despite never being trained on uncertainties of Level-2 and Level-3, achieved an $87 \pm 2\%$ success rate, and even 68% in the worst-case scenario. This demonstrates that its Lyapunov-based robustness loss (22) enables generalization to previously unseen local uncertainties, a critical capability for real-world systems even when some state spaces fall outside the solvability conditions.

F. 3-D Quadrotor

To validate the effectiveness of the neural network dynamics approximation and the corresponding controller on high-dimensional dynamic systems, we conducted hover control tests for a 12-dimensional 3-D quadrotor system. Our primary goal was to verify whether the Lyapunov conditions based on neural

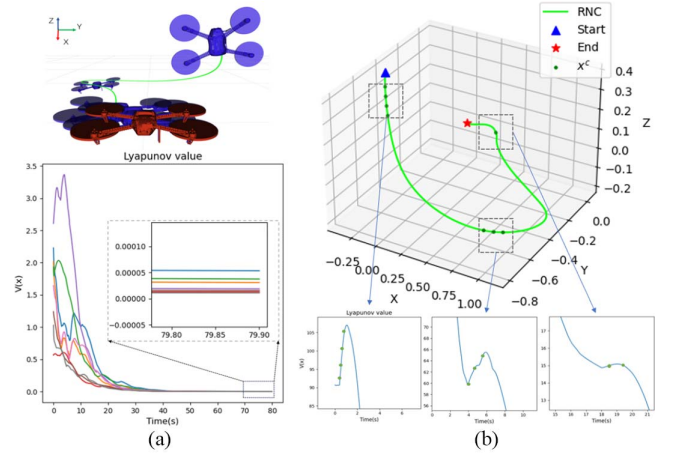


Fig. 13. (a) Snapshots of hovering experiment of a 3-D quadrotor real dynamic system which converge to the origin state (red), and value of the neural-network lyapunov functions along the simulated trajectories. (b) Green dots represent some *counter-examples* that violate the robustness condition, accounting for 4.75% of all states, and correspond to the segments where the Lyapunov value rises. (a) Hover; and (b) counter-examples.

network dynamics could ensure the robustness of the actual system.

Experimental results, obtained with the actual dynamics, show that under RNC control the Lyapunov value converges to zero. Nevertheless, Fig. 13(a) indicates that the decrease is not strictly monotonic, this behavior is primarily attributed to discrepancies between the actual dynamics and its neural network approximation. Unlike previous low-dimensional examples where the mean-squared error (MSE) was on the order of 10^{-6} , the 3-D quadrotor system exhibits a larger MSE of approximately 10^{-4} .

Capturing all nonlinear effects is particularly challenging in high-dimensional systems, especially in states that violating robustness conditions. In this example, as shown in Fig. 13(b), the proportion of *counter-examples* is 4.75%.

This increased modeling discrepancy and a small proportion of *counter-examples* lead to a temporary increase in the Lyapunov function within actual systems. Enhancing the neural network model's accuracy could improve robustness, but the controller still effectively stabilized the system to the equilibrium point with a high probability.

The initial selection of $\Delta_\sigma(x)$ and $\Delta_\epsilon(x)$ is guided by the necessity condition (50) for solvability. This condition requires the controller to compensate for system uncertainties, ensuring solvability of the Lyapunov-based control problem within the region of attraction. Excessively large $\Delta_\sigma(x)$ or $\Delta_\epsilon(x)$ may render the problem unsolvable, while overly small bounds would fail to account for real-world uncertainties. During the training process, we monitor the proportion of *counter-examples* to further refine these boundaries, typically maintaining this proportion below 5%. Consequently, $\Delta_\sigma(x)$ and $\Delta_\epsilon(x)$ are adjusted empirically, following the methodology applied in previous cases. This approach ensures both the existence of a feasible controller and subsequent system stabilization.

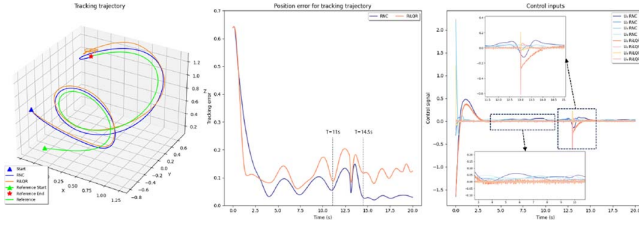


Fig. 14. Tracking performance of RNC versus RiLQR. The controllers follow a random trajectory starting with an initial offset. RiLQR shows larger deviations, notably between 11 and 14.5 s (a high-uncertainty zone), due to limited robustness. RNC effectively deals with these uncertainties, maintaining stability and lower tracking error throughout.

To fully evaluate the dynamic performance, we conducted trajectory tracking tests to compare RNC and RiLQR, as illustrated in Fig. 14. Quantitative metrics presented in Table II demonstrate RNC's superior performance: its root mean square error (RMSE, 0.08 ± 0.015) and mean absolute error (MAE, 0.06 ± 0.035) were 33.3% lower than those of RiLQR, its control variance (0.018 ± 0.03) was far smaller than that of RiLQR (0.276 ± 0.04).

As shown in Fig. 14, RiLQR exhibits a faster initial response but suffers from oscillatory behavior in later stages, consistent with the control signal trends observed in Figs. 4 and 6. Specifically, RNC requires more time to reach a steady state compared with RiLQR, as reflected in the convergence time metric T_c . This discrepancy stems from their distinct optimization strategies. RiLQR employs local linearization and iterative quadratic optimization, enabling rapid initial reactions but often neglecting global uncertainty. While the RNC learns a validated Lyapunov function to guarantee stability across the entire state space. While this approach inherently accounts for uncertainties and constraints, it results in a more conservative convergence rate.

While RNC exhibits slightly higher total energy consumption ($E_u = 187 \pm 20$) compared with RiLQR ($E_u = 150 \pm 30$), this trade-off is justified by its superior robustness and tracking stability. Notably, RNC achieves these gains while maintaining a comparable average speed. This enhanced stability is critical in high-dimensional systems, where even minor oscillations can precipitate catastrophic failure.

VIII. CONCLUSION AND FUTURE WORK

This article presents a robustness-guaranteed RL method for synthesizing a neural network controller to maintain system stability under bounded uncertainties in both dynamics modeling and state estimates. A set of robustness losses is proposed to train the controller and a coupled Lyapunov function to ensure robustness. The bilevel optimization is solved by the Gurobi solver since the maximization of the loss functions can be converted into Mixed-integer linear programming. Via HL-PWL representations, a necessary condition is proposed to ensure the solvability of the RL problem and is used to refine the attractive region. In addition, it is shown that the robustness capability can be boosted by amplifying the uncertainty to a derived maximum extent in training. The proposed method has been tested on the

simulated inverted pendulum and quadrotors, and its performance has been compared against existing methods, showing guaranteed robustness and applicability on high-dimensional systems.

One of the key challenges is scalability, as the performance and robustness of high-dimensional systems require further improvement. To address this, integrating system order reduction with uncertainty modeling could be a promising approach. Our work applies to systems that can be stabilized by locally Lipschitz bounded controllers, according to one of our assumptions. However, scenarios where boundaries are exceeded may result in failure. Furthermore, the strict robustness conditions demand a reduction in the Lyapunov value along system trajectories, which can lead to overly conservative solutions and dynamic responses. Future work will investigate extending this framework to achieve broader system applicability and stochastic robustness.

APPENDIX

A. HL-PWL Function Properties

Consider a rectangular compact attractive region in $\mathbb{R}^n$ of the form

$$\mathcal{D} \triangleq \{(x_1, \dots, x_n) : 0 \leq x_i \leq m_i \gamma, i \in \{1, \dots, n\}\} \quad (58)$$

where γ is the grid step and $m_i \in \mathbb{Z}_+$, and a The simplicial boundary configuration P defined by the set of hyperplanes

$$\{(\mathbf{x}) = x_q - k_q \gamma = 0\}, \{(\mathbf{x}) = x_i - (x_j - k_j \gamma) = 0\} \quad (59)$$

$$\{(\mathbf{x}) = (x_i - k_i \gamma) - x_j = 0\} \quad (60)$$

where $x \in \mathbb{R}^n$, $q \in \{1, \dots, n\}$, $k_q \in \{1, \dots, m_q - 1\}$, $k_j \in \{1, \dots, m_j - 1\}$, $k_i \in \{1, \dots, m_i - 1\}$, $i \in \{1, \dots, n - 1\}$, and $j \in \{i + 1, \dots, m_j - 1\}$, $\forall i \in \{1, \dots, n - 1\}$. The boundary configuration P' divides the domain $\mathcal{D}$ into a compact domain $\mathbf{S}$, $\text{PWL}_{H'}[\mathbf{S}]$ is a linear vector space and the set of all the continue PWL mappings $\mathcal{F} : \mathbf{S} \subset \mathbb{R}^n \mapsto \mathbb{R}^m$. The HL-PWL function defines a basis of $\text{PWL}_{H'}[\mathbf{S}]$ [46]. The basis function is obtained by nesting multiple absolute value functions. The construction of basis functions is based on the generating function, given as

$$\kappa_\zeta(x_i, x_j) = \frac{1}{4} \{ \zeta(\zeta(-x_i) + x_j) - \zeta(-x_i + \zeta(x_j)) + \zeta(-x_i) + \zeta(x_j) - \zeta(-x_i + x_j) \}. \quad (61)$$

In this article, using $\zeta(z) = |z|$ yields

$$\kappa(x_i, x_j) = \begin{cases} x_i, & \text{if } 0 \leq x_i \leq x_j \\ x_j, & \text{if } 0 \leq x_j \leq x_i \\ 0, & \text{if } x_i < 0 \text{ or } x_j < 0. \end{cases} \quad (62)$$

Then, by recursively composing κ , the basic function is generated. The recursive process is as follows:

$$\begin{cases} \kappa^0(x_r) = 1 \\ \kappa^1(x_r) = \kappa(x_r, x_r) \\ \kappa^2(x_{r_1}, x_{r_2}) = \kappa(x_{r_1}, x_{r_2}) \\ \vdots \\ \kappa^k(x_{r_1}, \dots, x_{r_k}) = \kappa(x_{r_1}, \kappa^{k-1}(x_{r_2}, \dots, x_{r_k})). \end{cases} \quad (63)$$

TABLE III
PHYSICAL PARAMETERS AND NETWORK SIZES IN SIMULATION

	Pendulum	2-D Quadrotor	3-D Quadrotor
Mass	1	0.486	0.468
Length ¹	1	0.25	0.225
Moment of inertia	0.33	0.00383	[4.9E-03, 4.9E-03, 8.8E-03]
$u_{equilibrium}$	0	2.38383	1.14777
Gravity	9.81	9.81	9.81
Forward dynamics network f	(5, 5)	(7, 7)	(10, 10)
Controller network u	(2, 2)	(6, 4)	(8, 6)
Lyapunov network V	(8, 4, 4)	(10, 10, 4)	(14, 8, 4)

Note: mass(kg), length(m), inertia(kg · m²), gravity(m/s²). ¹For quadrotors, length refers to the single robot arm.

The κ^k function has k testings of the absolute value functions and is given as

$$\kappa^k(x_{r_1}, \dots, x_{r_k}) = \begin{cases} x_{r_1}, & \text{if } 0 \leq x_{r_1} \leq \kappa^{k-1}(x_{r_2}, \dots, x_{r_k}) \\ \kappa^{k-1}(x_{r_2}, \dots, x_{r_k}), & \text{if } 0 \leq \kappa^{k-1}(x_{r_2}, \dots, x_{r_k}) \leq x_{r_1} \\ 0, & \text{if } x_{r_i} < 0 \text{ for some } i \in \{1, \dots, k\} \end{cases} \quad (64)$$

where $k \in \{1, \dots, n\}$, $r_1, r_2, \dots, r_n \in \{x_1, \dots, x_n\}$.

B. State Space Partitioning

The control function and Lyapunov function are both PWL functions and have the same domain of definition. The domain of the Lyapunov candidate is a compact set as

$$\mathcal{D} \triangleq \{(x_1, \dots, x_n) : |x| \leq |x_{\max}|\} \quad (65)$$

where $|x_{\max}| = r\gamma$, $r \in \mathbb{Z}_+$, and γ is the grid step.

To adequately represent the scheme of the HL-PWL representation, the attractive region $\mathcal{D}$ in (58) with $m_i = m$, $\forall i \in \{1, \dots, n\}$. In addition, 2^n different linear transformations are defined as $T_j : e_i \mapsto (-1)^{k_{i,j}} e_i$, here $k_{i,j} \in \{0, 1\}$, $\forall i \in \{1, \dots, n\}$, $j \in \{1, 2, \dots, 2^n\}$, and e_i is the i th unitary vector. Consider a set S_j of $\mathcal{D}$ under the mapping T_j , and its associated partition P_j resulting from the transformation of boundary points under T_j . With these definitions, it follows that $\mathcal{D} = \bigcup_{j=1}^{2^n} S_j$. The piecewise continuous mappings are given as $f \in \text{PWL}[\mathcal{D}]$, where

$$\text{PWL}[\mathcal{D}] = \text{span}\{\text{PWL}_{P_1}[S_1], \dots, \text{PWL}_{P_{2^n}}[S_{2^n}]\}. \quad (66)$$

C. Physics and Neural Network Implementation

The physical parameters and network sizes are summarized in Table III, where each network entry represents the size of the hidden layers. All networks in the results section utilize a leaky ReLU activation function.

DATA AVAILABILITY STATEMENT

The source code can be accessed via the GitHub repository at https://github.com/DuofengPan/RGRL_2025.

REFERENCES

- [1] S. S. Husain, M. Q. Kadhim, A. S. M. Al-Obaidi, A. F. Hasan, A. J. Humaidi, and D. N. Al Huseini, "Design of robust control for vehicle steer-by-wire system," *Indonesian J. Sci. Technol.*, vol. 8, no. 2, pp. 197–216, 2023.
- [2] A. M. Annaswamy, "Adaptive control and intersections with reinforcement learning," *Annu. Rev. Control, Robot., Auton. Syst.*, vol. 6, pp. 65–93, Jan. 2023.
- [3] J. Zhang, Q. Zhu, and W. Lin, "Neural stochastic control," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 9098–9110, Sep. 2022.
- [4] H. Komurcugil, S. Biricik, S. Bayhan, and Z. Zhang, "Sliding mode control: Overview of its applications in power converters," *IEEE Ind. Electron. Mag.*, vol. 15, no. 1, pp. 40–49, Mar. 2021.
- [5] S. A. Bortoff, P. Schwerdtner, C. Danielson, S. Di Cairano, and D. J. Burns, "H-infinity loop-shaped model predictive control with HVAC application," *IEEE Trans. Control Syst. Technol.*, vol. 30, no. 5, pp. 2188–2203, Sep. 2022.
- [6] M. Rashad, A. El-Badawy, and A. Hamza, "Robust h-infinity control for the longitudinal dynamics of a fixed wing aircraft," in *Proc. AIAA SCITECH 2025 Forum*, 2025, p. 2080.
- [7] I. Tzortzis, C. D. Charalambous, T. Charalambous, C. K. Kourtellis, and C. N. Hadjicostis, "Robust linear quadratic regulator for uncertain systems," in *Proc. IEEE 55th Conf. Decis. Control (CDC)*, 2016, pp. 1515–1520.
- [8] J. Hajar, T. Kargin, V. Malik, and B. Hassibi, "The distributionally robust infinite-horizon LQR," in *Proc. IEEE 63rd Conf. Decis. Control (CDC)*, 2024, pp. 6539–6546.
- [9] H. Nishimura, N. Mehr, A. Gaidon, and M. Schwager, "RAT iLQR: A risk auto-tuning controller to optimally account for stochastic model mismatch," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 763–770, Apr. 2021.
- [10] N. J. Kong, C. Li, G. Council, and A. M. Johnson, "Hybrid iLQR model predictive control for contact implicit stabilization on legged robots," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4712–4727, Dec. 2023.
- [11] C. C. Millan-Arias, B. J. Fernandes, F. Cruz, R. Dazeley, and S. Fernandes, "A robust approach for continuous interactive actor-critic algorithms," *IEEE Access*, vol. 9, pp. 104242–104260, 2021.
- [12] A. Mokhtari, A. Ozdaglar, and S. Pattathil, "A unified analysis of extra-gradient and optimistic gradient methods for saddle point problems: Proximal point approach," in *Proc. Int. Conf. Artif. Intell. Statist. (PMLR)*, 2020, pp. 1497–1507.
- [13] T. Oikarinen, W. Zhang, A. Megretski, L. Daniel, and T.-W. Weng, "Robust deep reinforcement learning through adversarial loss," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 26156–26167, Dec. 2021.
- [14] K.-H. Lai, D. Zha, Y. Li, and X. Hu, "Dual policy distillation," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020.
- [15] I. Ilahi et al., "Challenges and countermeasures for adversarial attacks on deep reinforcement learning," *IEEE Trans. Artif. Intell.*, vol. 3, no. 2, pp. 90–109, Apr. 2021.
- [16] L. Canese et al., "Multi-agent reinforcement learning: A review of challenges and applications," *Appl. Sci.*, vol. 11, no. 11, p. 4948, 2021.
- [17] G. A. White, C. D. Hill, F. A. Pollock, L. C. Hollenberg, and K. Modi, "Demonstration of non-markovian process characterisation and control on a quantum processor," *Nat. Commun.*, vol. 11, no. 1, p. 6301, 2020.
- [18] Y. Tian et al., "Evolutionary large-scale multi-objective optimization: A survey," *ACM Comput. Surv. (CSUR)*, vol. 54, no. 8, pp. 1–34, 2021.
- [19] J. G. Clement and C. Kroer, "First-order methods for Wasserstein distributionally robust MDP," in *Proc. Int. Conf. Mach. Learn. (PMLR)*, 2021, pp. 2010–2019.
- [20] G. Zhang and Q. Zhang, "Multiportfolio optimization with CVAR risk measure," *J. Data, Inf. Manage.*, vol. 1, no. 3, pp. 91–106, 2019.
- [21] J. Tobin, et al., "Domain randomization and generative models for robotic grasping," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 3482–3489.
- [22] X. He, W. Huang, and C. Lv, "Trustworthy autonomous driving via defense-aware robust reinforcement learning against worst-case observational perturbations," *Transp. Res. Part C: Emerg. Technol.*, vol. 163, 2024, Art. no. 104632.
- [23] D. Sun, A. Jamshidnejad, and B. De Schutter, "A novel framework combining MPC and deep reinforcement learning with application to freeway traffic control," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 6756–6769, Jul. 2024.

- [24] S. Gros and M. Zanon, "Data-driven economic NMPC using reinforcement learning," *IEEE Trans. Autom. Control*, vol. 65, no. 2, pp. 636–648, Feb. 2020.
- [25] M. H. Cohen, N. Csomay-Shanklin, W. D. Compton, T. G. Molnar, and A. D. Ames, "Safety-critical controller synthesis with reduced-order models," 2024, *arXiv:2411.16479*.
- [26] M. Han, L. Zhang, J. Wang, and W. Pan, "Actor-critic reinforcement learning for control with stability guarantee," *IEEE Robot. Automat. Lett.*, vol. 5, no. 4, pp. 6217–6224, Oct. 2020.
- [27] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. 18th Eur. Control Conf. (ECC)*, Piscataway, NJ, USA: IEEE Press, 2019, pp. 3420–3431.
- [28] S. Bo, B. T. Agyeman, X. Yin, and J. Liu, "Control invariant set enhanced safe reinforcement learning: Improved sampling efficiency, guaranteed stability and robustness," *Comput. Chem. Eng.*, vol. 179, 2023, Art. no. 108413.
- [29] F. Blanchini, E.-S. H. G. Giordano, and E. D. Sontag, "Control-theoretic methods for biological networks," in *Proc. IEEE Conf. Decis. Control (CDC)*, Piscataway, NJ, USA: IEEE Press, 2018, pp. 466–483.
- [30] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, "Safe exploration in continuous action spaces," 2018, *arXiv:1801.08757*.
- [31] A. Kumar and R. Sharma, "A stable Lyapunov constrained reinforcement learning based neural controller for non linear systems," in *Proc. Int. Conf. Comput., Communication Automat.*, 2015, pp. 185–189.
- [32] H. Dai, B. Landry, L. Yang, M. Pavone, and R. Tedrake, "Lyapunov-stable neural-network control," 2021, *arXiv:2109.14152*.
- [33] H. Zou, J. Feng, H. Zhao, and Y. Shi, "Analytical Lyapunov function discovery: An RL-based generative approach," 2025, *arXiv:2502.02014*.
- [34] H. Zhang, H. Chen, D. Boning, and C.-J. Hsieh, "Robust reinforcement learning on state observations with learned optimal adversary," 2021, *arXiv:2101.08452*.
- [35] Y. Wang and S. Zou, "Online robust reinforcement learning with model uncertainty," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 7193–7206, Dec. 2021.
- [36] T. Zhou, T. Guo, C. Dang, and M. Beer, "Bayesian reinforcement learning reliability analysis," *Computer Methods Appl. Mechan. Eng.*, vol. 424, 2024, Art. no. 116902.
- [37] E. Goan and C. Fookes, "Bayesian neural networks: An introduction and survey," in *Proc. Appl. Bayesian Data Sci.: CIRM Jean-Morlet Chair*, 2020, pp. 45–87.
- [38] Q. Liu, Q. Zhou, R. Yang, and J. Wang, "Robust representation learning by clustering with bisimulation metrics for visual reinforcement learning with distractions," *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 7, pp. 8843–8851, 2023.
- [39] G. An, S. Moon, J.-H. Kim, and H. O. Song, "Uncertainty-based offline reinforcement learning with diversified q-ensemble," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 7436–7447, Dec. 2021.
- [40] J. Morimoto and K. Doya, "Robust reinforcement learning," *Neural Comput.*, vol. 17, no. 2, pp. 335–359, 2005.
- [41] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken, "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function," *Neural Netw.*, vol. 6, no. 6, pp. 861–867, 1993.
- [42] J. Gawlikowski et al., "A survey of uncertainty in deep neural networks," *Artif. Intell. Rev.*, vol. 56 Suppl., no. 1, pp. 1513–1589, 2023.
- [43] Q. Long and C. Wu, "A nonsmooth equation system solver based on subgradient method," in *Proc. 22nd Int. Conf. Digit. Signal Process. (DSP)*, 2017, pp. 1–5.
- [44] H. Dai, B. Landry, M. Pavone, and R. Tedrake, "Counterexample guided synthesis of neural network lyapunov functions for piecewise linear systems," in *Proc. 59th IEEE Conf. Decis. Control (CDC)*, Piscataway, NJ, USA: IEEE Press, 2020, pp. 1274–1281.
- [45] X. Jiang, Y. Pang, X. Li, J. Pan, and Y. Xie, "Deep neural networks with elastic rectified linear units for object recognition," *Neurocomputing*, vol. 275, pp. 1132–1139, Sep. 2017.
- [46] P. Julian, A. Desages, and O. Agamennoni, "High-level canonical piecewise linear representation using a simplicial partition," *IEEE Trans. Circuits Syst. I: Fundam. Theory Appl.*, vol. 46, no. 4, pp. 463–480, Apr. 1999.
- [47] P. Julian, J. Guivant, and A. Desages, "A parametrization of piecewise linear Lyapunov functions via linear programming," *Int. J. Control*, vol. 72, nos. 7–8, pp. 702–715, 1999.